Hệ thống nhúng

Tuần 2 — Kiến trúc & tập lệnh vi điều khiển AVR

TS. Nguyễn Mạnh Hùng

Chương trình 9 tuần

Học phần: Hệ thống nhúng (Embedded Systems)

Mục tiêu buổi học

Chuẩn đầu ra liên quan

CLO2: Phân tích được kiến trúc phần cứng vi điều khiển và lập trình được bằng hợp ngữ/C.

Sau tuần 2, sinh viên có khả năng:

- Mô tả kiến trúc CPU AVR: ALU, tệp thanh ghi, SREG, ngăn xếp, bộ đếm chương trình.
- Giải thích chu trình Fetch–Decode–Execute và pipeline đơn giản của AVR.
- Trình bày vai trò các thanh ghi `DDRx`, `PORTx`, `PINx` và không gian địa chỉ I/O.
- Vận dụng các chế độ định địa chỉ (addressing modes) của AVR.
- Viết chương trình hợp ngữ/C để đọc nút nhấn và điều khiển LED.

Nội dung buổi học

Kiến trúc CPU AVR chi tiết

Cổng vào/ra và định địa chỉ

Tập lệnh AVR

Tổng kết chương

Thực hành — Tuần 2

Kiến trúc CPU ARM Cortex-M chi tiết

GPIO và định địa chỉ trên ARM Cortex-M

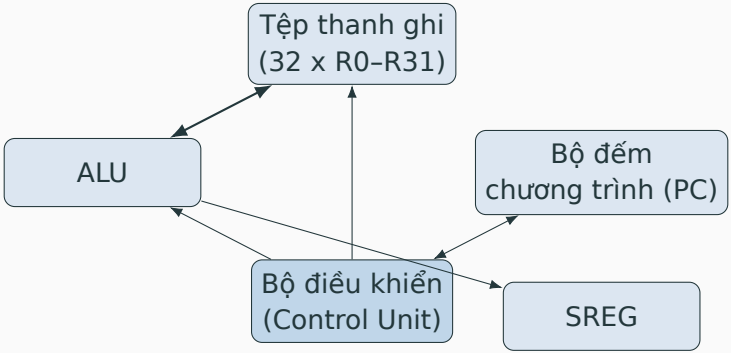
Tập lệnh Thumb-2 trên ARM Cortex-M

Thực hành — ARM Cortex-M (STM32)

Tổng kết & So sánh: AVR và ARM Cortex-M

Kiến trúc CPU AVR chi tiết

Sơ đồ khối chi tiết CPU AVR



CPU AVR gồm 4 khối chính (ALU, tệp thanh ghi, bộ đếm chương trình PC, thanh ghi trạng thái SREG), phối hợp bởi bộ điều khiển (Control Unit) giải mã và phát tín hiệu điều khiển.

Đơn vị số học – logic (ALU) và đường dữ liệu

- ALU (Arithmetic Logic Unit) thực hiện các phép toán số học (cộng, trừ) và logic (AND, OR, XOR, dịch/xoay bit).
- Kết nối **trực tiếp** với tệp 32 thanh ghi: đọc tối đa 2 toán hạng và ghi 1 kết quả trong cùng một chu kỳ xung nhịp.
- Sau mỗi phép toán, ALU cập nhật các cờ trạng thái trong thanh ghi SREG (Carry, Zero, Negative, Overflow...).

Bộ điều khiển (Control Unit) và giải mã lệnh

- Nhận mã lệnh (opcode) đã được lấy về từ bộ nhớ chương trình, **giải mã** để xác định thao tác cần thực hiện.
- Sinh các tín hiệu điều khiển: chọn thanh ghi nguồn/đích, chọn phép toán ALU, cho phép ghi bộ nhớ, cập nhật PC.
- Là ”bộ não” điều phối toàn bộ hoạt động của CPU theo từng chu kỳ xung nhịp.

Tệp thanh ghi (Register File) – tổng quan

- AVR có **32 thanh ghi làm việc đa dụng** 8-bit, ký hiệu `R0`–`R31`.
- Toàn bộ 32 thanh ghi kết nối trực tiếp với ALU $\rightarrow$ hầu hết lệnh chỉ cần 1 chu kỳ để đọc toán hạng, tính toán và ghi kết quả.
- Khác với kiến trúc dùng *thanh ghi tích luỹ* (accumulator) độc nhất, tệp thanh ghi lớn giúp giảm số lần truy cập bộ nhớ (data) chậm hơn.

Vì sao cần tới 32 thanh ghi?

- Kiến trúc RISC ”load–store”: ALU chỉ thao tác trên thanh ghi, **không** thao tác trực tiếp trên bộ nhớ SRAM.
- Càng nhiều thanh ghi $\rightarrow$ trình biên dịch càng ít phải nạp/lưu (load/store) dữ liệu ra SRAM $\rightarrow$ chương trình chạy nhanh hơn.
- Đánh đổi: mỗi thanh ghi cần 5 bit để định danh trong mã lệnh ($2^5 = 32$), chiếm nhiều không gian mã lệnh hơn.

Phân nhóm thanh ghi: R0–R15 và R16–R31

- **R0–R15**: chỉ dùng được với các lệnh hai thanh ghi (vd `MOV`, `ADD`, `CP`).
- **R16–R31**: dùng được với **mọi** lệnh, kể cả các lệnh có toán hạng *tức thời* (immediate) như `LDI`, `ANDI`, `ORI`, `SUBI`, `CPI`.
- Lý do: trường mã lệnh dành cho toán hạng tức thời chỉ đủ 4 bit để chọn thanh ghi $\rightarrow$ chỉ chọn được trong nhóm 16 thanh ghi cao.
- Hệ quả lập trình: nên ưu tiên dùng R16–R31 cho các biến cần nạp hằng số trực tiếp.

Ba thanh ghi con trỏ đặc biệt: X, Y, Z

Con trỏ	Byte cao	Byte thấp	Vai trò điển hình
X	R27	R26	Địa chỉ gián tiếp SRAM tổng quát
Y	R29	R28	Địa chỉ gián tiếp + dịch chuyển (khung ngăn xếp cục bộ)
Z	R31	R30	Địa chỉ gián tiếp + đọc bộ nhớ chương trình (LPM)

- 6 thanh ghi cao nhất (R26–R31) được ghép thành 3 cặp con trỏ 16-bit, dùng cho định địa chỉ gián tiếp.

Thanh ghi trạng thái SREG – cấu trúc 8 bit

Bit	Ký hiệu	Ý nghĩa
7	I	Global Interrupt Enable – cho phép ngắt toàn cục
6	T	Bit Copy Storage – lưu tạm 1 bit (dùng với BLD/BST)
5	H	Half Carry – nhớ nửa byte (dùng cho số BCD)
4	S	Sign – dấu hiệu, $S = N \oplus V$
3	V	Overflow – tràn số có dấu (bù hai)
2	N	Negative – kết quả âm (bit 7 = 1)
1	Z	Zero – kết quả bằng 0
0	C	Carry – nhớ/mượn từ phép toán

Cờ Carry (C) và ứng dụng

- Cờ **Carry (C)** = 1 khi phép cộng tràn ra ngoài bit 7 (không dấu), hoặc phép trừ cần mượn.
- Cho phép cộng/trừ số nhiều byte: `ADD` cho byte thấp, sau đó `ADC` (Add with Carry) cho byte cao lan truyền số nhớ.
- Cũng được cập nhật bởi các lệnh dịch/xoay bit (`LSL`, `LSR`, `ROL`, `ROR`).

Cờ Zero (Z) và ứng dụng

- Cờ **Zero (Z)** = 1 khi kết quả phép toán bằng 0.
- Là cờ được dùng nhiều nhất trong rẽ nhánh có điều kiện: sau lệnh `CP`/`CPI` (so sánh), `BREQ` kiểm tra Z=1, `BRNE` kiểm tra Z=0.
- Cũng dùng để kiểm tra vòng lặp đã đếm về 0 hay chưa sau lệnh `DEC`.

Cờ Negative (N) và Overflow (V)

- **Negative (N)**: phản ánh bit 7 (bit dấu) của kết quả – 1 nghĩa là kết quả âm (theo biểu diễn bù hai).
- **Overflow (V)**: bật khi kết quả phép toán *có dấu* vượt phạm vi biểu diễn được (vd cộng hai số dương ra kết quả âm).
- Chỉ dựa vào N **không đủ** để so sánh số có dấu chính xác khi có tràn – cần kết hợp N và V (xem cờ S).

Cờ dấu hiệu S và các cờ còn lại (H, T, I)

- **S = N XOR V**: cho kết quả so sánh *có dấu* đúng ngay cả khi có tràn số – các lệnh BRLT/BRGE dựa vào cờ này.
- **H** (Half Carry): hỗ trợ các phép toán số BCD (ít dùng trong lập trình C thông thường).
- **T** (Bit Copy Storage): vùng đệm 1 bit, nạp/xuất bằng BST/BLD – hữu ích khi cần ”di chuyển” một bit đơn lẻ.
- **I** (Interrupt Enable): bật/tắt bằng SEI/CLI, sẽ khai thác sâu ở Tuần 5 (Ngắt).

Ví dụ: trạng thái SREG sau lệnh ADD

Ví dụ

`R16 = 0xF0`, `R17 = 0x20`. Thực hiện `ADD R16, R17` $\rightarrow$ `R16 = 0x10` (tràn khỏi 8-bit).

- Kết quả không dấu $0xF0 + 0x20 = 0x110$ vượt 8 bit $\rightarrow$ **C = 1**.
- Kết quả `0x10` $\neq 0 \rightarrow$ **Z = 0**.
- Bit 7 kết quả = 0 $\rightarrow$ **N = 0**.
- Cộng hai số coi là ”âm” theo bù hai ra kết quả ”dương” $\rightarrow$ **V = 1**, do đó **S = N XOR V = 1**.

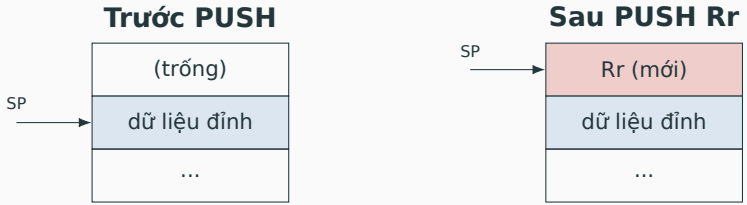
Con trỏ ngăn xếp (Stack Pointer)

- Cặp thanh ghi 16-bit SPH:SPL lưu **địa chỉ đỉnh ngăn xếp** hiện tại trong SRAM.
- Với ATmega328P (SRAM 2KB), chỉ cần khoảng 11–12 bit thấp là đủ để địa chỉ hoá toàn bộ SRAM.
- Chương trình khởi động (startup code) thường nạp SP bằng địa chỉ cuối SRAM (RAMEND) trước khi vào main().

Vùng nhớ Stack trong SRAM

- Ngăn xếp (stack) là một vùng của SRAM, **phát triển từ địa chỉ cao xuống địa chỉ thấp**.
- Dùng để lưu: địa chỉ trở về khi gọi hàm (CALL/RCALL), giá trị thanh ghi tạm (PUSH/POP), biến cục bộ của hàm (do trình biên dịch C quản lý).
- Nếu ngăn xếp ”lấn” vào vùng biến toàn cục/heap (stack overflow) $\rightarrow$ chương trình hoạt động sai không dự đoán được.

Hoạt động PUSH / POP trên Stack



PUSH Rr: ghi Rr vào ô nhớ tại SP, rồi giảm SP đi 1. POP Rd: tăng SP lên 1, rồi đọc dữ liệu vào Rd.

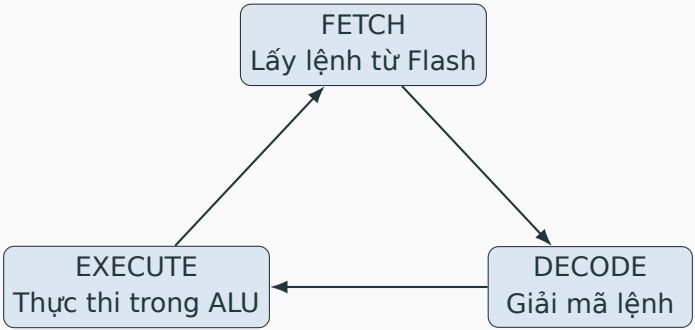
Stack và lời gọi hàm con: CALL / RET

- Lệnh CALL/RCALL: tự động **đẩy địa chỉ lệnh kế tiếp** (địa chỉ trở về) vào ngăn xếp, rồi nhảy tới hàm con.
- Lệnh RET: tự động **lấy địa chỉ trở về** từ đỉnh ngăn xếp, nạp lại vào PC.
- Đây là cơ chế phần cứng cho phép ngôn ngữ C hiện thực lời gọi hàm (function call) một cách tự nhiên.

Bộ đếm chương trình (Program Counter – PC)

- PC lưu **địa chỉ (theo từ lệnh)** của lệnh sẽ được thực thi tiếp theo trong bộ nhớ Flash.
- ATmega328P có 32KB Flash = 16K từ lệnh (16-bit/từ) $\rightarrow$ PC rộng **14 bit** ($2^{14} = 16384$).
- Mặc định PC tự động tăng sau mỗi lệnh; các lệnh nhảy/gọi hàm/rẽ nhánh sẽ ghi đè giá trị mới vào PC.

Chu trình Fetch – Decode – Execute trên AVR



Chu trình lặp lại liên tục: PC chỉ định lệnh cần lấy → giải mã xác định thao tác → thực thi và cập nhật thanh ghi/SREG/PC.

Minh hoạ Fetch – Decode – Execute qua ví dụ

Lệnh: `ADD R16, R17`

- **Fetch**: CPU đọc từ lệnh tại địa chỉ PC trong Flash, đưa vào thanh ghi lệnh.
- **Decode**: bộ điều khiển nhận diện đây là lệnh `ADD`, xác định toán hạng là R16 và R17.
- **Execute**: ALU cộng giá trị R16 và R17, ghi kết quả vào R16, cập nhật cờ trong SREG, PC tăng lên 1.

Chu kỳ lệnh và khái niệm CPI

- **CPI (Clock cycles Per Instruction)**: số chu kỳ xung nhịp trung bình để thực thi một lệnh.
- CPU CISC truyền thống: CPI thường > 1, có lệnh tốn hàng chục chu kỳ.
- AVR (RISC): phần lớn lệnh có **CPI** $\approx 1 \rightarrow$ ở 16 MHz, ATmega328P đạt thông lượng **lên tới 16 triệu lệnh/giây (16 MIPS)**.

Vì sao hầu hết lệnh AVR chỉ tốn 1 chu kỳ?

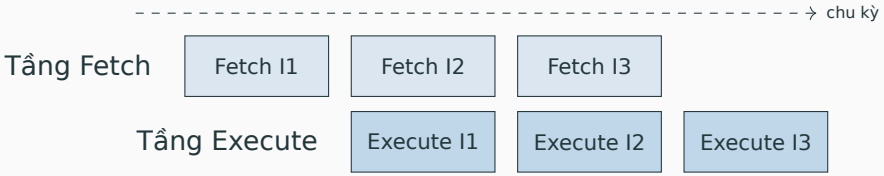
- Tập lệnh đơn giản, độ dài mã lệnh cố định (chủ yếu 16-bit/1 từ).
- ALU truy cập trực tiếp toàn bộ 32 thanh ghi, không cần các bước trung gian truy xuất bộ nhớ chậm.
- Áp dụng **pipeline đơn giản 2 tầng**: lệnh kế tiếp được ”lấy trước” (pre-fetch) trong khi lệnh hiện tại đang thực thi.

Bảng số chu kỳ theo loại lệnh

Loại lệnh	Số chu kỳ	Ví dụ
Số học/logic/dịch bit trên thanh ghi	1	`ADD`, `LSL`
Truy xuất SRAM qua con trỏ	2	`LD`, `ST`
Rẽ nhánh có điều kiện (nhảy/không nhảy)	1 hoặc 2	`BRNE`
Nhảy/gọi hàm tuyệt đối	3 hoặc 4	`JMP`, `CALL`

Ví dụ: ở 16 MHz, 1 chu kỳ $= 1/16\text{MHz} = 62{,}5$ ns $\rightarrow$ lệnh `CALL` (4 chu kỳ) tốn khoảng 250 ns.

Kiến trúc pipeline đơn giản 2 tầng



Trong khi lệnh I_1 đang được *thực thi*, lệnh I_2 đã được *lấy* (fetch) song song $\rightarrow$ thông lượng đạt gần 1 lệnh/chu kỳ dù mỗi lệnh thực chất cần 2 bước.

Lợi ích và giới hạn của pipeline 2 tầng

- **Lợi ích**: đơn giản, ít tốn phần cứng, đạt thông lượng gần 1 lệnh/chu kỳ cho hầu hết lệnh.
- **Giới hạn**: khi gặp lệnh nhảy/rẽ nhánh, lệnh đã ”lấy trước” có thể bị huỷ (pipeline bị ”xả rỗng”) $\rightarrow$ tốn thêm 1–2 chu kỳ.
- So với pipeline sâu (5–15 tầng) của ARM Cortex-A hay x86: AVR đơn giản hơn nhiều, dễ dự đoán thời gian thực thi – phù hợp hệ thời gian thực nhỏ.

So sánh kiến trúc AVR (RISC) với CISC truyền thống

Tiêu chí	RISC (AVR)	CISC (vd x86 cũ)
Số lượng lệnh	Ít, đơn giản (≈131)	Nhiều, phức tạp (hàng trăm)
Độ dài mã lệnh	Cố định (chủ yếu 16-bit)	Thay đổi (1–15 byte)
Truy xuất bộ nhớ	Chỉ qua LD/ST (load–store)	ALU thao tác trực tiếp trên bộ nhớ
Số chu kỳ/lệnh	Đa số 1 chu kỳ	Thường nhiều hơn 1, thay đổi

Kiểm tra nhanh – Câu hỏi

Thảo luận

1. Vì sao AVR có tới 32 thanh ghi đa dụng thay vì chỉ một thanh ghi tích luỹ (accumulator) như nhiều CPU 8-bit cũ?
2. Sau lệnh `SUB R16, R17` nếu `R16` = `R17` thì cờ nào trong `SREG` sẽ bằng 1?

Kiểm tra nhanh – Gợi ý đáp án

1. Kiến trúc RISC load–store hạn chế ALU thao tác trực tiếp trên bộ nhớ; nhiều thanh ghi giúp trình biên dịch/lập trình viên giữ nhiều biến trong CPU hơn, giảm số lần truy cập SRAM chậm, tăng tốc độ và giảm số lệnh cần thiết.
2. Kết quả R16 - R17 = 0 $\rightarrow$ cờ **Zero (Z) = 1**; cờ Carry (C) = 0 vì không cần mượn.

Tổng kết: Kiến trúc CPU AVR

- CPU AVR = ALU + 32 thanh ghi đa dụng + PC (14-bit) + SREG (8 cờ) + bộ điều khiển.
- SREG lưu các cờ C, Z, N, V, S, H, T, I – nền tảng cho rẽ nhánh có điều kiện và phép toán nhiều byte.
- Con trỏ ngăn xếp (SP) quản lý Stack – phục vụ PUSH/POP và lời gọi hàm CALL/RET.
- Nhờ pipeline 2 tầng đơn giản, đa số lệnh AVR thực thi trong **1 chu kỳ xung nhịp**.

Cổng vào/ra và định địa chỉ

Không gian địa chỉ I/O trong AVR

- AVR dành riêng **64 thanh ghi I/O** ”cổ điển” (địa chỉ I/O `0x00–0x3F`) cho các ngoại vi: GPIO, Timer, UART, ADC...
- Cùng các thanh ghi này cũng được ánh xạ vào không gian bộ nhớ dữ liệu tại địa chỉ `0x20–0x5F` (địa chỉ I/O + `0x20`).
- Nhờ vậy có thể truy cập thanh ghi ngoại vi bằng **hai cách**: lệnh I/O chuyên dụng hoặc lệnh truy xuất bộ nhớ thông thường.

Không gian I/O mở rộng (Extended I/O)

- ATmega328P có thêm vùng **Extended I/O** tại địa chỉ bộ nhớ `0x60–0xFF` (các ngoại vi ”sinh sau” như Timer1, SPI, TWI...).
- Vùng mở rộng **không** truy cập được bằng `IN/OUT` – chỉ truy cập được qua LD/ST/LDS/STS.
- Trình biên dịch C (avr-gcc) tự động chọn lệnh phù hợp – lập trình viên chỉ cần gán tên thanh ghi trong `avr/io.h`.

Hai cách truy cập thanh ghi I/O

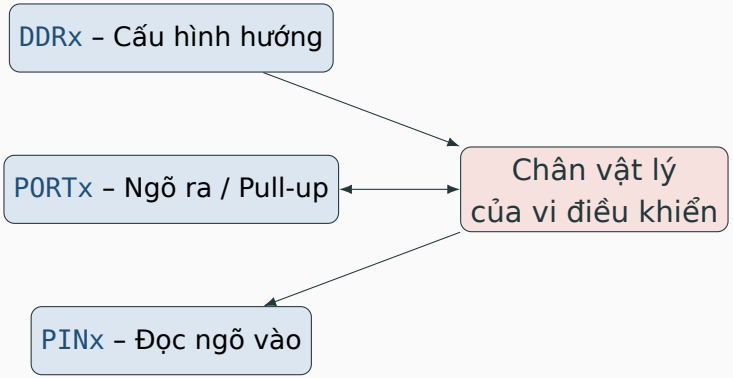
Cách truy cập	Đặc điểm	Phạm vi
`IN`/`OUT`	1 chu kỳ, địa chỉ I/O 6-bit trong mã lệnh	Chỉ 64 thanh ghi I/O cổ điển
LD/ST qua data space	2 chu kỳ, dùng địa chỉ bộ nhớ thông thường	Toàn bộ I/O + Extended I/O

Ghi chú: `SBI`/`CBI`/`SBIC`/`SBIS` chỉ thao tác được trên 32 thanh ghi I/O đầu tiên (địa chỉ I/O `0x00`–`0x1F`) do trường địa chỉ trong mã lệnh chỉ có 5 bit.

Ba cổng GPIO trên ATmega328P

- **PORTB**: 8 bit (`PB0`–`PB7`); trên Arduino Uno tương ứng D8–D13 (PB6/PB7 dùng cho thạch anh).
- **PORTC**: 7 bit khả dụng (`PC0`–`PC6`); tương ứng A0–A5 (PC6 thường dùng cho RESET).
- **PORTD**: 8 bit (`PD0`–`PD7`); tương ứng D0–D7 (PD0/PD1 dùng cho UART).

Ba thanh ghi điều khiển mỗi cổng



Thanh ghi DDRx (Data Direction Register)

- Mỗi bit `DDRxn` chọn hướng cho chân tương ứng: **1 = ngõ ra (output)**, **0 = ngõ vào (input)**.
- Vd: `DDRB |= (1 << PB0);` cấu hình chân PB0 làm ngõ ra, các bit khác của `DDRB` giữ nguyên.
- Sau reset, mặc định toàn bộ `DDRx` = 0 $\rightarrow$ mọi chân đều là ngõ vào.

Thanh ghi PORTx khi là ngõ ra

- Khi `DDRxn` = 1 (ngõ ra): bit `PORTxn` quyết định mức điện áp xuất ra chân – **1 = mức cao (5V)**, **0 = mức thấp (0V)**.
- Vd: `PORTB |= (1 << PB0);` đặt chân PB0 lên mức cao (bật LED nối đúng chiều).

Thanh ghi PORTx khi là ngõ vào

- Khi `DDRxn` = 0 (ngõ vào): bit `PORTxn` = 1 sẽ **bật điện trở kéo lên nội** (internal pull-up, khoảng 20–50 kΩ) cho chân đó.
- Điện trở kéo lên nội giúp chân đọc mức **cao** khi không có gì tác động, và mức **thấp** khi nút nhấn nối chân đó xuống GND – không cần điện trở ngoài.
- Vd: `PORTD |= (1 << PD2);` bật pull-up nội cho chân PD2 (đã cấu hình là ngõ vào).

Thanh ghi PINx – đọc ngõ vào

- `PINx` phản ánh **mức điện áp thực tế** đo được tại chân, bất kể chân đang là ngõ vào hay ngõ ra.
- Đọc trạng thái nút nhấn: kiểm tra bit tương ứng trong `PINx`, vd `if (PIND & (1 << PD2))`.
- Tính năng đặc biệt trên ATmega328P: **ghi 1** vào một bit của `PINx` sẽ *đảo* (toggle) giá trị `PORTxn` tương ứng – hữu ích để đảo trạng thái LED mà không cần đọc-sửa-ghi.

Bảng tổng hợp: DDRx / PORTx → trạng thái chân

DDRxn	PORTxn	Trạng thái chân
0	0	Ngõ vào, tổng trở cao (Hi-Z), không pull-up
0	1	Ngõ vào, có điện trở kéo lên nội (pull-up)
1	0	Ngõ ra, mức thấp (LOW)
1	1	Ngõ ra, mức cao (HIGH)

Ví dụ: cấu hình PB0 output, PB1 input

```c
#include <avr/io.h>

int main(void) {
    DDRB |= (1 << PB0);   // PB0 = output (vd dieu khien LED)
    DDRB &= ~(1 << PB1);  // PB1 = input  (vd doc nut nhan)
    PORTB |= (1 << PB1);  // Bat pull-up noi cho PB1

    while (1) {
        if (PINB & (1 << PB1))
            PORTB |= (1 << PB0);   // Bat LED
        else
            PORTB &= ~(1 << PB0);  // Tat LED
    }
}
```

Ví dụ tương đương bằng thư viện Arduino

- `pinMode(pin, OUTPUT)` / `pinMode(pin, INPUT)` ≡ thiết lập bit tương ứng trong `DDRx`.
- `pinMode(pin, INPUT_PULLUP)` ≡ `DDRxn`=0 và `PORTxn`=1 (bật pull-up nội).
- `digitalWrite(pin, HIGH/LOW)` ≡ ghi bit trong `PORTx`.
- `digitalRead(pin)` ≡ đọc bit trong `PINx`.

So sánh: thanh ghi trực tiếp và thư viện Arduino

Tiêu chí	Thanh ghi trực tiếp	Thư viện Arduino
Tốc độ	Nhanh nhất (1 lệnh `OUT`)	Chậm hơn (qua nhiều lớp hàm)
Dung lượng code	Nhỏ gọn	Lớn hơn
Tính di động	Gắn với từng chip cụ thể	Dễ chuyển giữa các board Arduino
Độ khó	Cần hiểu phần cứng	Dễ tiếp cận hơn

Tổng quan các chế độ định địa chỉ trong AVR

Chế độ định địa chỉ	Toán hạng	Ví dụ
Thanh ghi trực tiếp	Rd, Rr	`MOV R1,R2`
Tức thời (Immediate)	Rd, K	`LDI R16,0x05`
I/O trực tiếp	A, Rd	`IN R16,PORTB`
Bộ nhớ dữ liệu trực tiếp	k, Rd	`LDS R16,0x0150`
Bộ nhớ dữ liệu gián tiếp	X/Y/Z, Rd	`LD R16,X`
Gián tiếp có dịch chuyển	Y+q/Z+q, Rd	`LDD R16,Y+3`
Chương trình tương đối	k (12-bit có dấu)	`RJMP loop`
Chương trình tuyệt đối	k	`JMP loop`

Định địa chỉ tức thời (Immediate Addressing)

- Toán hạng là một **hằng số 8-bit** nằm ngay trong mã lệnh, không cần truy xuất thêm bộ nhớ.
- Các lệnh tiêu biểu: `LDI Rd,K` (nạp hằng số), `SUBI`, `ANDI`, `ORI`, `CPI` (so sánh với hằng số).
- Giới hạn: chỉ áp dụng được cho `R16`–`R31` (trường thanh ghi đích chỉ 4 bit trong mã lệnh).

Định địa chỉ thanh ghi trực tiếp (Direct Register)

- Toán hạng nguồn và đích đều là thanh ghi, được chỉ định trực tiếp bằng chỉ số (5 bit/thanh ghi, đủ cho 32 thanh ghi).
- Ví dụ: `MOV Rd,Rr` (sao chép), `ADD Rd,Rr`, `EOR Rd,Rr`.
- Nhanh nhất về mặt giải mã – không cần tính toán địa chỉ bộ nhớ.

Định địa chỉ trực tiếp bộ nhớ dữ liệu

- Lệnh `LDS Rd,k` / `STS k,Rr`: địa chỉ SRAM 16-bit được nhúng **nguyên vẹn** trong từ lệnh thứ hai.
- Vì cần thêm 1 từ lệnh cho địa chỉ, đây là lệnh **32-bit** (2 từ, 2 chu kỳ nạp lệnh).
- Phù hợp truy cập biến toàn cục có địa chỉ cố định, đã biết tại thời điểm biên dịch.

Định địa chỉ gián tiếp qua con trỏ X/Y/Z

- Địa chỉ SRAM cần truy cập được lưu **trong một cặp thanh ghi con trỏ** (X, Y hoặc Z), có thể thay đổi được lúc chạy.
- Ví dụ: `LD Rd,X` đọc byte tại địa chỉ đang lưu trong X vào Rd; `ST X,Rr` ghi ngược lại.
- Thiết yếu cho: duyệt mảng, thao tác chuỗi ký tự, truyền tham số con trỏ trong C.

Các biến thể định địa chỉ gián tiếp

Cú pháp	Ý nghĩa	Con trỏ sau lệnh
`LD Rd,X`	Đọc tại địa chỉ X	Không đổi
`LD Rd,X+`	Đọc tại X, rồi tăng con trỏ	X = X+1
`LD Rd,-X`	Giảm con trỏ trước, rồi đọc	X = X−1
`LD Rd,Y+q`	Đọc tại địa chỉ (Y+q)	Không đổi
`LPM Rd,Z`	Đọc 1 byte từ Flash tại Z	Không đổi

Cú pháp tương tự áp dụng cho Y, Z và cho lệnh ST (ghi).

Định địa chỉ có dịch chuyển (Displacement)

- Lệnh `LDD Rd,Y+q` / `STD Y+q,Rr`: truy cập ô nhớ tại địa chỉ **Y cộng thêm độ dịch q** (0–63), **không làm thay đổi** giá trị Y.
- Rất hữu ích khi truy cập nhiều trường trong một cấu trúc (struct) hoặc biến cục bộ trên khung ngăn xếp (stack frame) mà không cần cập nhật con trỏ liên tục.
- Trình biên dịch C thường dùng Y làm ”con trỏ khung ngăn xếp” (frame pointer) và phát sinh LDD/STD để truy cập biến cục bộ.

Tự tăng / tự giảm con trỏ

- **Tự tăng sau** (post-increment, X+): đọc/ghi trước, sau đó con trỏ tăng thêm 1 – lý tưởng để duyệt mảng *tiến*.
- **Tự giảm trước** (pre-decrement, -X): con trỏ giảm 1 trước, sau đó mới đọc/ghi – lý tưởng để duyệt mảng *lùi* hoặc cài đặt ngăn xếp phần mềm.
- Giúp thay thế 2 lệnh (truy xuất + cập nhật con trỏ bằng lệnh riêng) chỉ bằng 1 lệnh duy nhất.

Ví dụ: duyệt mảng SRAM bằng con trỏ Z

```
    ldi  ZL, low(mang)   ; nap dia chi thap cua mang vao Z
    ldi  ZH, high(mang)  ; nap dia chi cao cua mang vao Z
    ldi  r17, 5           ; so phan tu can duyet
    clr  r18               ; bien tong = 0

sum_loop:
    ld   r16, Z+           ; doc phan tu, Z tu dong tang
    add  r18, r16           ; cong don vao r18
    dec  r17
    brne sum_loop            ; lap den khi het mang
```

Mỗi lần lặp: đọc 1 phần tử mảng và tự động dịch con trỏ Z sang phần tử kế tiếp – không cần lệnh cộng con trỏ riêng.

So sánh tốc độ các chế độ định địa chỉ

Chế độ định địa chỉ	Số chu kỳ điển hình
Thanh ghi trực tiếp / Tức thời / I/O trực tiếp	1
Bộ nhớ gián tiếp (LD/ST qua X/Y/Z)	2
Gián tiếp có dịch chuyển (LDD/STD)	2
Bộ nhớ dữ liệu trực tiếp (LDS/STS)	2

Truy cập thanh ghi luôn nhanh hơn truy cập SRAM – một lý do quan trọng để tối ưu hoá chương trình nhúng bằng cách giữ biến ”nóng” trong thanh ghi.

Kiểm tra nhanh – Câu hỏi

Thảo luận

1. Muốn chân PC3 là ngõ vào có điện trở kéo lên nội, cần thiết lập `DDRC` và `PORTC` như thế nào?
2. Chế độ định địa chỉ nào phù hợp nhất để duyệt qua một mảng 20 phần tử trong SRAM? Vì sao?

Kiểm tra nhanh – Gợi ý đáp án

1. Cần `DDRC` bit 3 = 0 (ngõ vào) và `PORTC` bit 3 = 1 (bật pull-up nội), ví dụ: `DDRC &= ~(1<<PC3); PORTC |= (1<<PC3);`
2. Định địa chỉ gián tiếp có tự tăng (X+/Y+/Z+): mỗi lệnh `LD Rd,Z+` vừa đọc phần tử vừa tự động dịch con trỏ sang phần tử kế tiếp, không tốn thêm lệnh cộng địa chỉ.

Tổng kết: Cổng vào/ra và định địa chỉ

- Mỗi cổng GPIO có 3 thanh ghi: `DDRx` (hướng), `PORTx` (ngõ ra/pull-up), `PINx` (đọc vào).
- Không gian I/O gồm 64 thanh ghi cổ điển (`IN`/`OUT`) và vùng mở rộng chỉ truy cập qua `LD`/`ST`.
- AVR hỗ trợ nhiều chế độ định địa chỉ: tức thời, thanh ghi trực tiếp, bộ nhớ trực tiếp, gián tiếp qua X/Y/Z (có/không dịch chuyển, tự tăng/giảm).
- Chọn đúng chế độ định địa chỉ giúp chương trình vừa ngắn gọn vừa nhanh hơn.

Tập lệnh AVR

Tổng quan tập lệnh AVR

- Tập lệnh AVR có khoảng **131 lệnh**, hầu hết dài **16-bit (1 từ lệnh)**.
- Một số lệnh cần không gian địa chỉ lớn (`JMP`, `CALL`, `LDS`, `STS`) dài **32-bit (2 từ lệnh)**.
- Tập lệnh tương đối nhỏ nhưng đủ mạnh để trình biên dịch C sinh mã hiệu quả – lập trình viên nhúng vẫn cần hiểu để tối ưu hoá khi cần.

Định dạng lệnh (Instruction Format)

- Mỗi từ lệnh 16-bit gồm: trường **mã thao tác (opcode)** và các trường **toán hạng** (chỉ số thanh ghi, hằng số, địa chỉ…).
- Lệnh hai thanh ghi (vd `ADD Rd,Rr`) dùng 5 bit cho mỗi thanh ghi – đủ địa chỉ hoá cả 32 thanh ghi.
- Lệnh có toán hạng tức thời (vd `LDI Rd,K`) dùng 4 bit chọn thanh ghi (chỉ `R16`–`R31`) và 8 bit cho hằng số K.
- `JMP`/`CALL`/`LDS`/`STS` cần thêm 1 từ lệnh 16-bit chứa địa chỉ/hằng số đầy đủ.

Phân loại nhóm lệnh AVR – tổng quan

Nhóm lệnh	Chức năng chính	Ví dụ
Số học & Logic	Tính toán trên thanh ghi	`ADD`, `AND`
Dịch/xoay bit	Thao tác từng bit	`LSL`, `ROR`
So sánh & rẽ nhánh	Điều khiển luồng có điều kiện	`CPI`, `BRNE`
Nhảy & gọi hàm	Điều khiển luồng không điều kiện	`JMP`, `CALL`
Thao tác bit I/O đơn	Điều khiển/kiểm tra 1 bit thanh ghi I/O	`SBI`, `SBIC`
Di chuyển dữ liệu	Trao đổi dữ liệu thanh ghi $\leftrightarrow$ bộ nhớ	`MOV`, `LDS`
Điều khiển hệ thống	Điều khiển CPU, ngắt, năng lượng	`SEI`, `SLEEP`

Nhóm lệnh số học: ADD, ADC, SUB, SUBI, SBC

- `ADD Rd,Rr`: $Rd \leftarrow Rd + Rr$.
- `ADC Rd,Rr`: cộng có nhớ (Add with Carry) – dùng để cộng số nhiều byte.
- `SUB Rd,Rr`: $Rd \leftarrow Rd - Rr$. `SUBI Rd,K`: trừ hằng số (chỉ R16–R31).
- `SBC Rd,Rr`: trừ có mượn (Subtract with Carry) – dùng cho số nhiều byte.
- Tất cả cập nhật đầy đủ các cờ C, Z, N, V, S, H trong SREG.

Nhóm lệnh logic: AND, OR, EOR, COM, NEG

- `AND Rd,Rr` / `ANDI Rd,K`: AND theo bit – thường dùng để ”xoá” (clear) các bit cụ thể.
- `OR Rd,Rr` / `ORI Rd,K`: OR theo bit – thường dùng để ”bật” (set) các bit cụ thể.
- `EOR Rd,Rr`: XOR theo bit – `EOR Rd,Rd` là cách nhanh nhất để đưa thanh ghi về 0.
- `COM Rd`: đảo tất cả các bit (bù một). `NEG Rd`: đổi dấu theo bù hai ($Rd \leftarrow 0 - Rd$).

Ví dụ: ADD/SUB và ảnh hưởng lên cờ

Phép toán	Kết quả	Cờ SREG bị ảnh hưởng
`0x05 + 0x03`	`0x08`	Z=0, C=0, N=0, V=0
`0xFF + 0x01`	`0x00`	Z=1, C=1, N=0, V=0
`0x05 - 0x05`	`0x00`	Z=1, C=0, N=0, V=0
`0x00 - 0x01`	`0xFF`	Z=0, C=1, N=1, V=0

Quan sát: `0xFF + 0x01` tràn khỏi 8-bit nên **C=1**; `0x00 - 0x01` cần mượn nên **C=1** và kết quả mang dấu âm (bù hai) nên **N=1**.

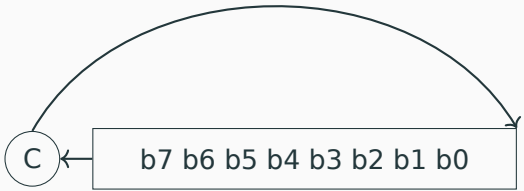
Nhóm lệnh tăng/giảm: INC, DEC

- `INC Rd`: $Rd \leftarrow Rd + 1$. `DEC Rd`: $Rd \leftarrow Rd - 1$.
- **Không** ảnh hưởng cờ Carry (khác với `ADD`/`SUB`) – vì mục đích chính là đếm vòng lặp, không phải cộng số nhiều byte.
- Rất thường gặp trong vòng lặp `for`: biến đếm giảm dần kết hợp `DEC` + `BRNE` để kiểm tra đã về 0 hay chưa.

Nhóm lệnh dịch bit: LSL, LSR

- LSL Rd (Logical Shift Left): dịch trái 1 bit, bit 7 cũ đưa vào cờ C, bit 0 mới = 0. Tương đương nhân 2 (không dấu).
- LSR Rd (Logical Shift Right): dịch phải 1 bit, bit 0 cũ đưa vào cờ C, bit 7 mới = 0. Tương đương chia 2 (không dấu).
- Nhanh hơn nhiều so với dùng lệnh nhân/chia (AVR cơ bản không có phép chia phần cứng).

Nhóm lệnh xoay bit: ROL, ROR



ROL Rd: xoay trái qua cờ Carry

- ROL Rd: dịch trái, bit 7 cũ $\rightarrow$ C, giá trị C *cũ* $\rightarrow$ bit 0 mới.
- ROR Rd: dịch phải, bit 0 cũ $\rightarrow$ C, giá trị C *cũ* $\rightarrow$ bit 7 mới.
- Cho phép dịch bit nối tiếp qua nhiều byte (ví dụ số 16-bit): LSL byte thấp rồi ROL byte cao để lan truyền carry.

Ứng dụng dịch/xoay bit

- **Nhân/chia nhanh cho 2**: dịch bit rẻ hơn nhiều lần so với lệnh nhân/chia đầy đủ.
- **Dịch số nhiều byte**: dùng LSL/R0L liên tiếp qua các byte, cờ Carry đóng vai trò ”cầu nối” giữa các byte.
- **Giao tiếp nối tiếp kiểu bit-banging**: gửi/nhận từng bit qua chân GPIO bằng R0L/R0R kết hợp kiểm tra cờ C (mô phỏng phần cứng UART/SPI bằng phần mềm).

Nhóm lệnh so sánh: CP, CPI, CPC

- `CP Rd,Rr`: tính `Rd` − `Rr` **chỉ để cập nhật cờ**, **không** thay đổi `Rd`.
- `CPI Rd,K`: so sánh thanh ghi (R16–R31) với hằng số.
- `CPC Rd,Rr`: so sánh có mượn (Compare with Carry) – dùng khi so sánh số nhiều byte, nối tiếp sau một lệnh `CP` của byte thấp hơn.
- Luôn được dùng **ngay trước** một lệnh rẽ nhánh có điều kiện.

Nhóm lệnh rẽ nhánh có điều kiện

Lệnh	Điều kiện nhảy	Thường dùng sau
BRNE	Z = 0 (khác nhau)	CP/CPI
BREQ	Z = 1 (bằng nhau)	CP/CPI
BRLT	Nhỏ hơn (có dấu)	CP/CPI
BRGE	Lớn hơn hoặc bằng (có dấu)	CP/CPI

Rẽ nhánh mở rộng: BRCS, BRCC

- BRCS (Branch if Carry Set): nhảy nếu C=1 – dùng để kiểm tra ”nhỏ hơn” khi so sánh số **không dấu**.
- BRCC (Branch if Carry Clear): nhảy nếu C=0 – ”lớn hơn hoặc bằng” khi so sánh số không dấu.
- **Lưu ý quan trọng**: so sánh số có dấu dùng BRLT/BRGE (dựa trên cờ S); so sánh số không dấu dùng BRCS/BRCC (dựa trên cờ C) – dùng nhầm dẫn tới lỗi logic tinh vi, khó phát hiện.

Ví dụ: vòng lặp dùng so sánh & rẽ nhánh

```
    ldi  r16, 0          ; bien dem i = 0
    ldi  r17, 10          ; gioi han lap = 10

loop:
    cpi  r16, 10           ; so sanh i voi 10
    brne skip_end           ; neu i != 10 thi tiep tuc
    rjmp loop_done            ; neu i == 10 thi thoat vong lap

skip_end:
    ; ... than vong lap ...
    inc  r16                 ; i = i + 1
    rjmp loop

loop_done:
    nop
```

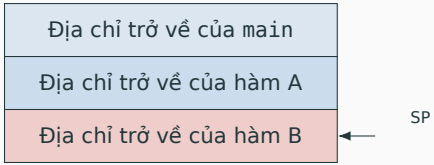
Nhóm lệnh nhảy không điều kiện: RJMP, JMP

- `RJMP label`: nhảy **tương đối** so với PC hiện tại, phạm vi ±2K từ lệnh (12-bit có dấu) – 2 chu kỳ.
- `JMP label`: nhảy **tuyệt đối** tới bất kỳ địa chỉ nào trong Flash – 3 chu kỳ, tốn 2 từ lệnh.
- Lưu ý: JMP/CALL chỉ tồn tại trên các dòng AVR có bộ nhớ chương trình lớn hơn 8K từ lệnh (như ATmega328P); các chip AVR nhỏ hơn chỉ có RJMP/RCALL.

Nhóm lệnh gọi/trở về hàm con

- `RCALL label` / `CALL label`: đẩy địa chỉ trở về vào ngăn xếp rồi nhảy tới hàm con (tương đối/tuyệt đối tương tự `RJMP`/`JMP`).
- `RET`: lấy địa chỉ trở về từ ngăn xếp, nạp lại vào PC – kết thúc hàm con.
- `RETI`: giống `RET` nhưng dùng để kết thúc **trình phục vụ ngắt** (ISR) – tự động bật lại cờ `I` (Global Interrupt Enable).

Ngăn xếp trong lời gọi hàm lồng nhau



`main` gọi hàm A (`CALL`), hàm A gọi hàm B (`CALL`) – ngăn xếp lưu lần lượt 2 địa chỉ trở về; mỗi lệnh RET lấy ra theo đúng thứ tự ngược lại (LIFO – Last In, First Out).

Nhóm lệnh thao tác bit đơn trên I/O: SBI, CBI

- `SBI A,b`: đặt (set) bit b của thanh ghi I/O tại địa chỉ A lên 1 – vd `SBI PORTB,0`.
- `CBI A,b`: xoá (clear) bit b về 0 – vd `CBI PORTB,0`.
- Chỉ tốn **2 chu kỳ**, thao tác trực tiếp trên 1 bit mà không ảnh hưởng các bit khác – nhanh hơn nhiều so với đọc-sửa-ghi cả thanh ghi bằng `IN/ANDI/OUT`.
- Giới hạn: chỉ áp dụng cho 32 thanh ghi I/O đầu tiên (địa chỉ I/O 0–31).

Nhóm lệnh kiểm tra bit và bỏ qua lệnh kế

- `SBIC A,b` (Skip if Bit in I/O register is Cleared): bỏ qua lệnh kế tiếp nếu bit = 0.
- `SBIS A,b` (Skip if Bit in I/O register is Set): bỏ qua lệnh kế tiếp nếu bit = 1.
- Cho phép cài đặt logic điều kiện **không cần rẽ nhánh riêng biệt** – rất phổ biến khi chờ (poll) trạng thái một chân hoặc cờ ngoại vi.

Ví dụ SBIC: chờ trạng thái nút nhấn

```
wait_press:
    sbic PIND, PD2   ; bo qua lenh ke neu PD2 = 0 (dang nhan)
    rjmp wait_press    ; PD2 = 1 (chua nhan) -> tiep tuc cho

    ; toi day nghia la nut da duoc nhan (PD2 = 0)
    sbi  PORTB, PB0     ; bat LED
```

Vòng lặp `wait_press` liên tục kiểm tra bit PD2 của `PIND`; chương trình chỉ thoát vòng lặp khi nút được nhấn.

Nhóm lệnh di chuyển thanh ghi – thanh ghi

- `MOV Rd,Rr`: sao chép nội dung 1 thanh ghi 8-bit.
- `MOVW Rd,Rr`: sao chép **một cặp** thanh ghi (16-bit) chỉ trong 1 chu kỳ – hữu ích khi sao chép con trỏ.
- `LDI Rd,K`: nạp hằng số 8-bit trực tiếp (chỉ R16–R31).

Nhóm lệnh truy xuất SRAM

- `LD Rd,X/Y/Z` (và các biến thể +/-): đọc SRAM qua con trỏ gián tiếp.
- `ST X/Y/Z,Rr`: ghi SRAM qua con trỏ gián tiếp.
- `LDS Rd,k` / `STS k,Rr`: đọc/ghi SRAM tại địa chỉ 16-bit cố định, nhúng thẳng trong mã lệnh.
- Đây chính là nhóm lệnh duy nhất trong AVR được phép truy cập bộ nhớ dữ liệu (đúng triết lý RISC load–store).

Bảng tổng hợp: lệnh di chuyển dữ liệu

Lệnh	Cú pháp	Số chu kỳ
MOV	MOV Rd,Rr	1
MOVW	MOVW Rd,Rr	1
LDI	LDI Rd,K	1
LD/ST	LD Rd,X / ST X,Rr	2
LDD/STD	LDD Rd,Y+q	2
LDS/STS	LDS Rd,k	2
PUSH/POP	PUSH Rr / POP Rd	2

Nhóm lệnh điều khiển hệ thống

- `NOP`: không làm gì cả, tốn đúng 1 chu kỳ – dùng để canh thời gian chính xác.
- `SEI` / `CLI`: bật/tắt cờ `I` – cho phép/cấm toàn bộ ngắt (chi tiết ở Tuần 5).
- `SLEEP`: đưa CPU vào chế độ tiết kiệm năng lượng theo cấu hình thanh ghi `SMCR`.
- `WDR` (Watchdog Reset): ”nạp lại” bộ đếm watchdog để tránh bị reset ngoài ý muốn (chi tiết ở Tuần 5).

Ví dụ chương trình đầy đủ – Cộng hai số

```
.include "m328Pdef.inc"

.org 0x0000
    rjmp main

main:
    ldi  r16, 0x1E    ; so hang thu nhat = 30
    ldi  r17, 0x0C     ; so hang thu hai = 12
    add  r16, r17       ; r16 = r16 + r17 = 42

    ldi  r18, 0xFF        ; cau hinh PORTB la output
    out  DDRB, r18
    out  PORTB, r16        ; xuat ket qua ra PORTB (quan sat qua LED)

end:
    rjmp end                ; dung tai day
```

Ví dụ: vòng lặp đếm giảm dần

```
.include "m328Pdef.inc"

.org 0x0000
    rjmp main

main:
    ldi  r16, 0xFF     ; PORTB la ngo ra
    out  DDRB, r16
    ldi  r17, 200        ; bien dem = 200

count_loop:
    out  PORTB, r17        ; xuat gia tri dem hien tai ra LED
    dec  r17                 ; giam bien dem
    brne count_loop           ; lap lai neu r17 != 0

stop:
    rjmp stop                   ; dung tai day
```

Bảng thời gian thực thi lệnh theo nhóm

Nhóm lệnh	Số chu kỳ	Ví dụ
Số học/logic/dịch bit trên thanh ghi	1	`ADD`, `LSL`
Di chuyển thanh ghi, tức thời	1	`MOV`, `LDI`
Truy xuất SRAM qua con trỏ/trực tiếp	2	`LD`, `LDS`
Thao tác bit I/O	2	`SBI`, `CBI`
Kiểm tra & bỏ qua lệnh	1–3	`SBIC`, `SBIS`
Rẽ nhánh có điều kiện	1–2	`BRNE`
Nhảy/gọi tương đối	2–3	`RJMP`, `RCALL`
Nhảy/gọi tuyệt đối	3–4	`JMP`, `CALL`
Trở về hàm/ngắt	4	`RET`, `RETI`

Ước lượng hiệu năng thực tế

Bài toán

Vòng lặp `count_loop` ở ví dụ trước (`OUT` + `DEC` + `BRNE`) lặp 200 lần ở tần số 16 MHz. Ước lượng thời gian thực thi.

- 1 chu kỳ $= 1/16.10^{6} = 62{,}5$ ns.
- Mỗi vòng lặp: `OUT` (1) + `DEC` (1) + `BRNE` (2, vì nhảy) ≈ 4 chu kỳ.
- Tổng: $200 \times 4 = 800$ chu kỳ $\times 62{,}5$ ns $\approx 50\,\mu$s – **quá nhanh để mắt người nhìn thấy nhấp nháy**, cần vòng lặp lồng nhau để tạo trễ (delay) đáng kể.

Quan hệ mã C và assembly do compiler sinh ra

Mã C

```c
uint8_t a = 5, b = 3, c;
c = a + b;
```

Assembly (minh hoạ)

```
ldi  r24, 5
ldi  r25, 3
add  r24, r25
mov  r18, r24
```

avr-gcc cấp phát thanh ghi tự động; mã thực tế phụ thuộc mức tối ưu hoá (`-O0` đến `-O3`).

Ví dụ đối chiếu: vòng lặp for

```
for (i = 0; i < 10; i++) {
    PORTB = i;
}
```

```
    ldi  r16, 0        ; i = 0
loop:
    out  PORTB, r16      ; PORTB = i
    inc  r16               ; i++
    cpi  r16, 10             ; so sanh i voi 10
    brne loop                  ; lap neu i != 10
```

So sánh: Assembly viết tay và trình biên dịch C

Tiêu chí	Assembly viết tay	C qua trình biên dịch
Kiểm soát phần cứng	Tối đa, từng chu kỳ	Gián tiếp qua trình biên dịch
Tốc độ phát triển	Chậm, dễ lỗi	Nhanh hơn nhiều
Hiệu năng	Có thể tối ưu tối đa (thủ công)	Rất tốt với -02, gần bằng tay nghề trung bình
Khả năng bảo trì	Khó đọc, khó mở rộng	Dễ đọc, dễ tái sử dụng

Thực tế: phần lớn firmware viết bằng C; assembly chỉ dùng cho các đoạn cực kỳ nhạy thời gian hoặc khởi động hệ thống (startup).

Kiểm tra nhanh – Câu hỏi

Thảo luận

1. Vì sao `SBI`/`CBI` nhanh hơn cách dùng `IN` + `ORI`/`ANDI` + `OUT` để bật/tắt 1 bit?
2. Cho đoạn C: `while (i != 0) i--;` – đoán 2 lệnh assembly AVR tương ứng cho thân vòng lặp.

Kiểm tra nhanh – Gợi ý đáp án

1. `SBI`/`CBI` chỉ tốn 2 chu kỳ và 1 lệnh, thao tác thẳng trên 1 bit của thanh ghi I/O; cách dùng `IN`+`ORI`/`ANDI`+`OUT` cần 3 lệnh (đọc, sửa cả byte, ghi lại) – chậm hơn và tốn nhiều mã hơn.
2. Tương ứng `DEC r16` (giảm `i`) và `BRNE loop` (quay lại nếu `i` chưa về 0) – đúng mẫu hình `DEC`+`BRNE` đã học.

Tổng kết: Tập lệnh AVR

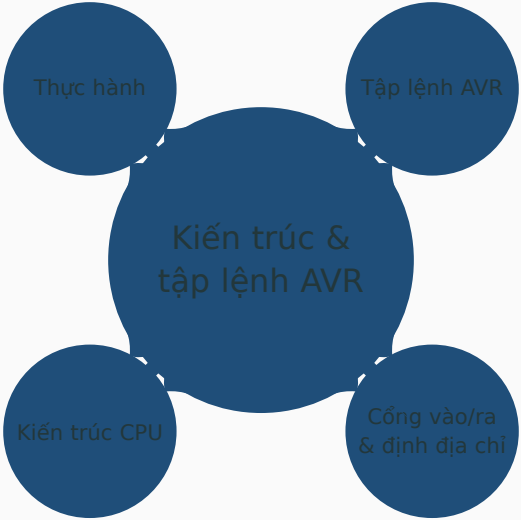
- Tập lệnh AVR (≈131 lệnh) chia thành 7 nhóm: số học/logic, dịch/xoay bit, so sánh/rẽ nhánh, nhảy/gọi hàm, thao tác bit I/O, di chuyển dữ liệu, điều khiển hệ thống.
- Phần lớn lệnh tốn 1–2 chu kỳ; nhảy/gọi hàm tuyệt đối tốn 3–4 chu kỳ.
- `SBIC`/`SBIS` và `SBI`/`CBI` là công cụ mạnh để thao tác I/O nhanh, gọn.
- Hiểu tập lệnh giúp đọc được mã assembly do trình biên dịch C sinh ra, phục vụ debug và tối ưu hoá.

Tổng kết chương

Tổng kết nội dung chính Tuần 2

- **Kiến trúc CPU AVR**: ALU, 32 thanh ghi đa dụng, SREG (8 cờ), con trỏ ngăn xếp, PC, chu trình Fetch–Decode–Execute, pipeline 2 tầng.
- **Cổng vào/ra & định địa chỉ**: thanh ghi `DDRx`/`PORTx`/`PINx`, không gian I/O, các chế độ định địa chỉ.
- **Tập lệnh AVR**: 7 nhóm lệnh, ví dụ chương trình hợp ngữ hoàn chỉnh, quan hệ với mã C.

Bản đồ tư duy – Tuần 2



Lộ trình các tuần tiếp theo

- **Tuần 3**: Lập trình C cho hệ thống nhúng.
- **Tuần 4**: Bộ nhớ, Bus dữ liệu và tối ưu hoá.
- **Tuần 5**: Ngắt, Timer/Counter và Watchdog.
- **Tuần 6–8**: Giao tiếp nối tiếp, RTOS, điều khiển & tổng kết.

Chuẩn đầu ra đã đạt được

CLO2 – Hoàn thành

Sinh viên đã có khả năng phân tích kiến trúc CPU AVR, tổ chức I/O và định địa chỉ, đồng thời đọc/viết được chương trình hợp ngữ AVR cơ bản.

Nền tảng hợp ngữ này giúp hiểu sâu hơn cách trình biên dịch C sinh mã máy – nội dung trọng tâm của Tuần 3.

Tài liệu tham khảo – Tuần 2

- Ryan Robucci, *CMPE 311 – AVR Architecture*, *AVR Ports*, *AVR CPU Registers* (Lecture notes), UMBC.
- Phillip Koopman, *18-348 – L03/L04: Instruction Sets*, CMU.
- Microchip/Atmel, *ATmega328P Datasheet* và *AVR Instruction Set Manual*.
- James K. Peckol, *Embedded Systems: A Contemporary Design Tool*, 2nd Ed., Wiley, 2019.

Thực hành — Tuần 2

Mục tiêu buổi thực hành

- Đấu nối và đọc đúng trạng thái một nút nhấn bằng điện trở kéo lên.
- Viết được chương trình đọc nút nhấn bằng cả hợp ngữ và C.
- Hoàn thành bài Lab: LED sáng theo trạng thái nút nhấn, có áp dụng chống dội phím.
- Làm quen kỹ năng debug cơ bản: mô phỏng, step-by-step, theo dõi thanh ghi.

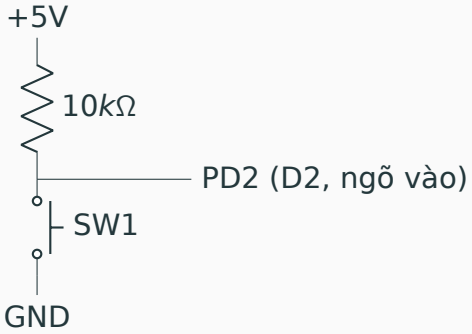
Chuẩn bị: linh kiện cần thiết

- Board Arduino Uno (ATmega328P), cáp USB.
- Breadboard và dây cắm (jumper wires).
- 1 nút nhấn (push button 4 chân), 1 LED, điện trở 330Ω (giới hạn dòng LED).
- Điện trở 10kΩ (tuỳ chọn, nếu không dùng pull-up nội của vi điều khiển).

Ôn lại: kết nối board Arduino Uno với máy tính

- Đã thực hiện chi tiết ở Tuần 1: cắm USB, chọn đúng *Board* và *Port* trong IDE.
- Tuần này dùng lại đúng thiết lập đó – chỉ thay đổi mạch đấu nối và mã nguồn.
- Nếu dùng Microchip Studio: cần chọn đúng *Device* (ATmega328P) và công cụ nạp (programmer) tương ứng.

Sơ đồ đấu nối nút nhấn với điện trở kéo lên



Khi chưa nhấn: PD2 đọc mức **cao** (kéo lên qua điện trở). Khi nhấn: PD2 nối xuống GND, đọc mức **thấp**.

Điện trở kéo lên: ngoài (external) và nội (internal)

Tiêu chí	Pull-up ngoài	Pull-up nội (`PORTx`)
Linh kiện thêm	Cần điện trở rời (vd 10kΩ)	Không cần
Cấu hình	Cố định trên mạch	Bật/tắt bằng phần mềm
Giá trị điện trở	Tuỳ chọn chính xác	Cố định trong chip (≈20–50kΩ)

Trong bài Lab hôm nay, ta dùng **pull-up nội** (`PORTD |= (1<<PD2);`) để đơn giản hoá mạch, không cần điện trở ngoài.

Đọc trạng thái nút nhấn – Code assembly

```
.include "m328Pdef.inc"

    ldi  r16, 0xFF
    out  DDRB, r16        ; PORTB la output (hien thi)
    cbi  DDRD, PD2          ; PD2 la input
    sbi  PORTD, PD2           ; bat pull-up noi cho PD2

read_loop:
    in   r16, PIND              ; doc toan bo PORTD
    out  PORTB, r16               ; xuat ra PORTB de quan sat
    rjmp read_loop
```

Đọc trạng thái nút nhấn – Code C

```c
#include <avr/io.h>

int main(void) {
    DDRB = 0xFF;               // PORTB la output
    DDRD &= ~(1 << PD2);       // PD2 la input
    PORTD |= (1 << PD2);       // bat pull-up noi

    while (1) {
        PORTB = PIND;          // doc PORTD, xuat ra PORTB
    }
}
```

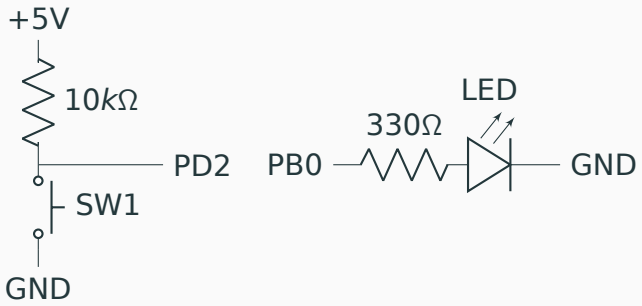
Lab: LED sáng khi nhấn nút – Đề bài

Yêu cầu

Nối nút nhấn vào chân PD2 (D2, dùng pull-up nội) và LED vào chân PB0 (D8, qua điện trở 330Ω). Viết chương trình: **khi nhấn nút thì LED sáng, khi thả nút thì LED tắt**.

- Có thể dùng thanh ghi trực tiếp (`DDRx`/`PORTx`/`PINx`) hoặc thư viện Arduino (`pinMode()`, `digitalRead()`, `digitalWrite()`).
- Nộp báo cáo: mã nguồn (C hoặc assembly) + video/ảnh chụp kết quả hoạt động.

Lab: LED sáng khi nhấn nút – Sơ đồ đấu nối



Nút nhấn (pull-up nội, không cần điện trở ngoài nếu dùng `PORTD`) bên trái; LED giới hạn dòng bởi điện trở 330Ω bên phải.

Lab: LED sáng khi nhấn nút – Code mẫu

```c
#include <avr/io.h>

int main(void) {
    DDRB |= (1 << PB0);      // D8 = PB0: output cho LED
    DDRD &= ~(1 << PD2);     // D2 = PD2: input cho nut nhan
    PORTD |= (1 << PD2);     // bat pull-up noi cho PD2

    while (1) {
        if (!(PIND & (1 << PD2)))  // muc thap = dang nhan
            PORTB |= (1 << PB0);    // bat LED
        else
            PORTB &= ~(1 << PB0);   // tat LED
    }
}
```

Hiện tượng dội phím và kỹ thuật chống dội (debounce)

- **Dội phím (switch bounce)**: tiếp điểm cơ khí rung nhiều lần (vài ms) trước khi ổn định khi nhấn/thả nút – vi điều khiển có thể đọc nhầm thành nhiều lần nhấn liên tiếp.
- **Chống dội bằng phần mềm**: sau khi phát hiện thay đổi trạng thái, chờ (delay) khoảng 20–50ms rồi đọc lại để xác nhận.
- **Chống dội bằng phần cứng**: thêm tụ điện nhỏ (vd 100nF) song song với nút nhấn để lọc nhiễu, hoặc dùng IC đệm có ngõ vào Schmitt trigger.

Sử dụng bộ mô phỏng (simulator) trong Microchip Studio

- Microchip Studio tích hợp sẵn bộ mô phỏng phần mềm (Simulator) – chạy thử chương trình **không cần board thật**.
- Cho phép quan sát giá trị thanh ghi, bộ nhớ, cờ SREG thay đổi theo từng lệnh.
- Hữu ích để kiểm tra logic chương trình assembly trước khi nạp vào phần cứng thực tế.

Debug từng bước và theo dõi thanh ghi

- **Step Into** (F11): thực thi từng lệnh một, *đi vào bên trong* lời gọi hàm/hàm con.
- **Step Over** (F10): thực thi từng dòng, nhưng chạy trọn vẹn hàm con mà *không* đi sâu vào bên trong.
- **Breakpoint** (F9): đặt điểm dừng tại một dòng lệnh cụ thể để chương trình tạm dừng khi chạy tới đó.
- Cửa sổ **Watch**/**Processor Status**: theo dõi trực tiếp giá trị thanh ghi `R0`–`R31`, `SREG`, `PC`, `SP` khi debug.

Sử dụng mạch nạp / In-Circuit Debugger thực tế

- Arduino Uno mặc định nạp chương trình qua **bootloader** (USB–Serial), không hỗ trợ debug phần cứng trực tiếp.
- Để debug thật trên chip: cần mạch nạp/debug ngoài như **AVR ISP mkII**, **Atmel-ICE**, hoặc **USBasp**, kết nối qua đầu nối ISP (6 chân) hoặc giao diện **debugWIRE** (qua chân RESET).
- Cho phép đặt breakpoint và xem thanh ghi **ngay trên chip thật**, chính xác hơn mô phỏng phần mềm.

Bài tập về nhà

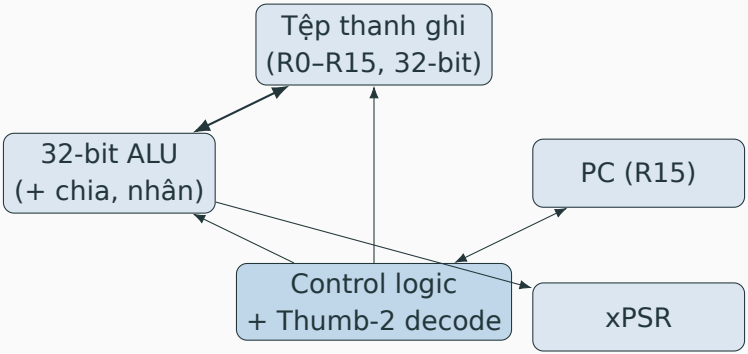
1. Đọc thêm 1 nút nhấn thứ hai (PD3), điều khiển thêm 1 LED thứ hai (PB1) độc lập với LED thứ nhất.
2. Viết lại chương trình Lab bằng hợp ngữ (assembly) thay vì C; so sánh số dòng lệnh và dung lượng file `.hex` sinh ra.
3. Thêm kỹ thuật chống dội phím (debounce) bằng `_delay_ms(20)` vào chương trình Lab, quan sát sự khác biệt khi nhấn nhả liên tục nhanh.

Tiêu chí đánh giá buổi thực hành

Tiêu chí	Điểm
Đọc đúng trạng thái nút nhấn (assembly hoặc C)	2
Bài Lab LED theo nút nhấn chạy đúng yêu cầu	4
Có áp dụng chống dội phím (debounce)	2
Báo cáo rõ ràng, đầy đủ	1
Hoàn thành đúng thời gian	1

Kiến trúc CPU ARM Cortex-M chi tiết

Sơ đồ khối chi tiết CPU ARM Cortex-M



CPU Cortex-M gồm 4 khối chính tương tự AVR (ALU, tệp thanh ghi, PC, thanh ghi trạng thái xPSR), nhưng độ rộng dữ liệu là **32-bit** thay vì 8-bit, và khối giải mã xử lý tập lệnh Thumb-2 hỗn hợp 16/32-bit.

Đơn vị số học – logic (ALU) trên Cortex-M

- ALU 32-bit thực hiện các phép toán số học (cộng, trừ) và logic (AND, OR, XOR, dịch/xoay bit) trên toán hạng **32-bit** (so với 8-bit của AVR).
- Từ Cortex-M3 trở lên: tích hợp sẵn **bộ chia phần cứng** (hardware divider) và **bộ nhân 32-bit một chu kỳ** (single-cycle multiplier) – AVR cơ bản không có bộ chia phần cứng.
- Sau mỗi phép toán, ALU cập nhật các cờ trạng thái trong thanh ghi APSR (một phần của xPSR): Carry, Zero, Negative, Overflow.

Bộ điều khiển và giải mã lệnh Thumb-2

- Nhận lệnh đã được lấy về từ bộ nhớ Flash, **giải mã** lệnh Thumb (16-bit) hoặc Thumb-2 (32-bit) để xác định thao tác.
- Sinh tín hiệu điều khiển: chọn thanh ghi nguồn/đích, chọn phép toán ALU, cho phép truy cập bộ nhớ qua D-bus, cập nhật PC.
- Khác biệt AVR: độ dài lệnh **thay đổi** (16 hoặc 32-bit) thay vì cố định 16-bit – bộ giải mã phức tạp hơn để nhận diện đúng ranh giới lệnh.

Tệp thanh ghi (Register File) trên Cortex-M – tổng quan

- Cortex-M có **16 thanh ghi** nhìn thấy được ở chế độ Thread: `R0`–`R12` đa dụng, `R13` (SP), `R14` (LR), `R15` (PC) – mỗi thanh ghi rộng **32-bit**.
- Ít thanh ghi hơn AVR (16 so với 32) nhưng mỗi thanh ghi chứa được số liệu **lớn hơn 4 lần** (32-bit so với 8-bit).
- Cũng là kiến trúc **load–store**: ALU chỉ thao tác trên thanh ghi, không thao tác trực tiếp trên bộ nhớ – giống triết lý RISC của AVR.

Vì sao Cortex-M chỉ cần 16 thanh ghi (không phải 32 như AVR)?

- Mỗi thanh ghi 32-bit đã đủ chứa trọn một địa chỉ bộ nhớ hoặc một số nguyên `int` tiêu chuẩn – không cần ghép cặp nhiều thanh ghi như AVR (X/Y/Z ghép từ 6 thanh ghi 8-bit).
- Trường định danh thanh ghi trong lệnh Thumb 16-bit chỉ có **3 hoặc 4 bit** (tuỳ dạng lệnh) – càng nhiều thanh ghi thì mã lệnh 16-bit càng khó biểu diễn gọn.
- Đánh đổi: ít thanh ghi hơn có thể khiến trình biên dịch phải nạp/lưu (spill) ra ngăn xếp nhiều hơn khi hàm có nhiều biến cục bộ.

Phân nhóm thanh ghi: Low registers và High registers

- **Low registers (R0–R7)**: truy cập được bởi **mọi** lệnh Thumb 16-bit – tương tự vai trò của R16–R31 trong AVR (nhóm được ưu tiên cho lệnh gọn).
- **High registers (R8–R12, cùng SP/LR/PC)**: chỉ một số lệnh 16-bit hạn chế thao tác được (vd `MOV`, `ADD`, `CMP`); muốn dùng đầy đủ phải chuyển sang mã 32-bit (Thumb-2).
- Lý do: trường thanh ghi 3-bit trong nhiều lệnh Thumb 16-bit chỉ đủ địa chỉ hoá 8 thanh ghi ($2^3 = 8$) – giống lý do AVR giới hạn lệnh tức thời cho R16–R31 (trường 4-bit).

Ba thanh ghi đặc biệt: SP, LR, PC

Thanh ghi	Tên đầy đủ	Vai trò điển hình
R13 (SP)	Stack Pointer	Con trỏ ngăn xếp – thực chất có **hai phiên bản** MSP (Main SP) và PSP (Process SP), chỉ một phiên bản hoạt động tại một thời điểm
R14 (LR)	Link Register	Tự động lưu địa chỉ trở về khi gọi hàm bằng `BL` – thay thế vai trò của ngăn xếp trong `RCALL`/`CALL` của AVR
R15 (PC)	Program Counter	Địa chỉ lệnh kế tiếp; bit 0 luôn = 1 để báo hiệu chế độ Thumb

- Khác biệt quan trọng so với AVR: **LR là một thanh ghi thật**, không cần đẩy địa chỉ trở về vào ngăn xếp ngay lập tức – chỉ hàm lồng nhau mới cần `PUSH {LR}`.

Thanh ghi trạng thái xPSR – cấu trúc

Thành phần	Vị trí bit	Ý nghĩa
APSR	31–27	Cờ số học: N (Negative), Z (Zero), C (Carry), V (Overflow) – tương ứng N, Z, C, V của SREG
IPSR	8–0	Số hiệu ngoại lệ/ngắt đang xử lý (0 nếu đang ở Thread mode)
EPSR	các bit khác	Trạng thái thực thi Thumb, trạng thái khối IT (rẽ nhánh có điều kiện mở rộng)

xPSR gộp chung vai trò của SREG (cờ số học) **và** một phần vai trò của NVIC (số hiệu ngắt) trong cùng một thanh ghi 32-bit.

Cờ Carry (C) trên Cortex-M và ứng dụng

- Cờ **Carry (C)** trong APSR = 1 khi phép cộng tràn ra ngoài bit 31 (không dấu), hoặc phép trừ cần mượn – vai trò giống hệt cờ C của AVR nhưng ở biên 32-bit thay vì 8-bit.
- Cho phép cộng/trừ số có độ rộng lớn hơn 32-bit: `ADDS` cho từ thấp, `ADCS` (Add with Carry) cho từ cao.
- Cũng được cập nhật bởi các lệnh dịch bit (LSLS, LSRS).

Cờ Zero (Z) trên Cortex-M và ứng dụng

- Cờ **Zero (Z)** = 1 khi kết quả phép toán bằng 0 – vai trò giống hệt AVR.
- Sau lệnh `CMP` (so sánh), `BEQ` kiểm tra Z=1, `BNE` kiểm tra Z=0 – tương ứng trực tiếp với `BREQ`/`BRNE` của AVR.
- Trên Cortex-M, hầu hết lệnh số học 16-bit **mặc định cập nhật cờ** (hậu tố S, vd `ADDS`); lệnh 32-bit trong khối `IT` có thể không cập nhật cờ – điểm khác biệt tinh vi so với AVR (AVR luôn cập nhật cờ với lệnh số học).

Cờ Negative (N) và Overflow (V) trên Cortex-M

- **Negative (N)**: phản ánh bit 31 (bit dấu) của kết quả 32-bit – giống AVR nhưng ở bit 31 thay vì bit 7.
- **Overflow (V)**: bật khi kết quả phép toán *có dấu* vượt phạm vi biểu diễn 32-bit.
- Cortex-M **không có cờ S** (Sign) riêng như AVR – lệnh rẽ nhánh có dấu (BLT, BGE) tính trực tiếp từ tổ hợp N và V theo mã điều kiện phần cứng, không cần XOR thủ công.

Ví dụ: trạng thái APSR sau lệnh ADDS

Ví dụ

`R0 = 0xFFFFFFF0`, `R1 = 0x00000020`. Thực hiện `ADDS R0, R0, R1` $\rightarrow$ `R0 = 0x00000010` (tràn khỏi 32-bit).

- Kết quả không dấu vượt 2^{32} $\rightarrow$ **C = 1**.
- Kết quả `0x10` $\neq 0$ $\rightarrow$ **Z = 0**.
- Bit 31 kết quả = 0 $\rightarrow$ **N = 0**.
- Cộng số âm (bù hai) với số dương ra kết quả dương hợp lệ trong phạm vi $\rightarrow$ **V = 0** (khác ví dụ AVR 8-bit vì phạm vi biểu diễn lớn hơn nhiều).

Con trỏ ngăn xếp kép: MSP và PSP

- Cortex-M có **hai** con trỏ ngăn xếp 32-bit: `MSP` (Main Stack Pointer, dùng mặc định và trong xử lý ngắt) và `PSP` (Process Stack Pointer, thường dùng riêng cho tác vụ trong RTOS).
- AVR chỉ có **một** con trỏ ngăn xếp `SPH:SPL` dùng chung cho mọi ngữ cảnh – đây là khác biệt kiến trúc quan trọng phục vụ hệ điều hành thời gian thực.
- Giá trị MSP ban đầu được **nạp tự động từ địa chỉ 0x00000000** ngay khi reset (đã trình bày ở tài liệu tham khảo về quy trình biên dịch/khởi động).

Vùng nhớ Stack trong SRAM trên Cortex-M

- Ngăn xếp là một vùng của SRAM, **phát triển từ địa chỉ cao xuống địa chỉ thấp** – giống hệt nguyên lý ở AVR.
- Dùng để lưu: địa chỉ trở về khi hàm lồng nhau (LR được đẩy vào ngăn xếp), giá trị thanh ghi tạm (PUSH/POP), biến cục bộ.
- Điểm khác: khi có ngắt xảy ra, phần cứng NVIC **tự động** đẩy 8 thanh ghi (R0–R3, R12, LR, PC, xPSR) vào ngăn xếp – AVR không có cơ chế lưu ngữ cảnh tự động tương đương, phần mềm ISR phải tự PUSH thủ công.

Hoạt động PUSH / POP trên Stack (Cortex-M)



`PUSH {Rlist}`: ghi các thanh ghi (mỗi thanh ghi 4 byte) vào SRAM, giảm SP tương ứng. `POP {Rlist}`: đọc ngược lại, tăng SP. Có thể đẩy/lấy **nhiều thanh ghi cùng lúc** trong 1 lệnh – AVR chỉ PUSH/POP được từng thanh ghi 8-bit một.

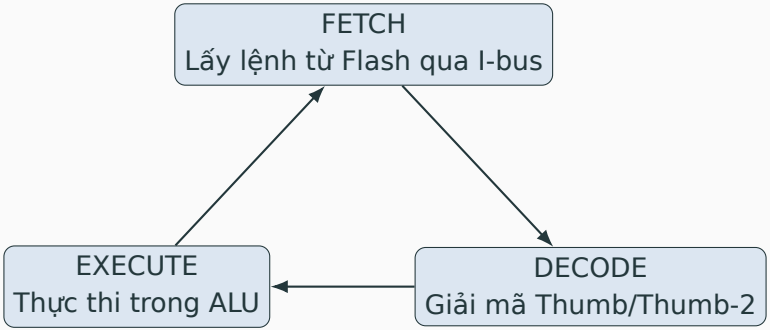
Stack và lời gọi hàm con: BL / BX LR

- Lệnh `BL label` (Branch with Link): nhảy tới hàm con, đồng thời **tự động ghi địa chỉ trở về vào LR** (thanh ghi, không phải ngăn xếp).
- Lệnh `BX LR`: nhảy về địa chỉ chứa trong LR – kết thúc hàm con.
- Với hàm gọi hàm lồng nhau, hàm ngoài phải tự `PUSH {LR}` trước khi gọi hàm trong (vì LR sẽ bị ghi đè bởi lệnh BL tiếp theo), rồi `POP {PC}` khi kết thúc – so với AVR, nơi *mọi* lệnh `CALL` đều tự động dùng ngăn xếp.

Bộ đếm chương trình (PC / R15) trên Cortex-M

- PC lưu **địa chỉ byte đầy đủ 32-bit** của lệnh kế tiếp trong Flash – khác AVR (PC lưu địa chỉ theo *từ lệnh* 16-bit, chỉ 14-bit cho ATmega328P).
- Không gian địa chỉ Flash trên STM32F103C8 (64KB–128KB Flash) chỉ dùng một phần nhỏ của dải 32-bit – PC có thể định địa chỉ tới 4GB về mặt lý thuyết.
- Bit 0 của PC luôn phải bằng 1 khi ghi (báo hiệu chế độ Thumb) dù lệnh luôn được nạp tại địa chỉ chẵn – vi phạm quy tắc này sẽ gây lỗi `HardFault` vì Cortex-M **chỉ hỗ trợ Thumb**.

Chu trình Fetch – Decode – Execute trên Cortex-M



Cùng nguyên lý ba giai đoạn như AVR, nhưng hiện thực bằng **pipeline 3 tầng** (thay vì 2 tầng của AVR) và nạp lệnh qua kiến trúc Harvard biến thể (I-bus/D-bus tách biệt trên cùng bản đồ địa chỉ).

Minh hoạ Fetch – Decode – Execute qua ví dụ

Lệnh: `ADDS R0, R0, R1`

- **Fetch**: CPU đọc từ lệnh (16-bit) tại địa chỉ PC trong Flash qua I-bus.
- **Decode**: khối Thumb & Thumb-2 decode nhận diện đây là lệnh `ADDS`, xác định toán hạng `R0` và `R1`.
- **Execute**: ALU cộng giá trị `R0` và `R1`, ghi kết quả vào `R0`, cập nhật cờ APSR, PC tăng thêm 2 (vì lệnh 16-bit).

Chu kỳ lệnh và khái niệm DMIPS/MHz

- Cortex-M3 đạt trung bình **1,25 DMIPS/MHz** (Dhrystone MIPS trên mỗi MHz) – thước đo hiệu năng chuẩn hoá theo tần số, tương tự khái niệm CPI nhưng đo ở mức chương trình chuẩn.
- Ở 72 MHz (STM32F103 điển hình), CPU đạt thông lượng khoảng **90 triệu lệnh Dhrystone/giây**, gấp nhiều lần AVR 16 MHz/16 MIPS dù xung nhịp ”chỉ” cao hơn 4,5 lần.
- Chênh lệch hiệu năng đến từ độ rộng dữ liệu 32-bit, pipeline sâu hơn, và bộ nhân/chia phần cứng.

Vì sao Cortex-M đạt CPI gần 1 dù tập lệnh phong phú hơn?

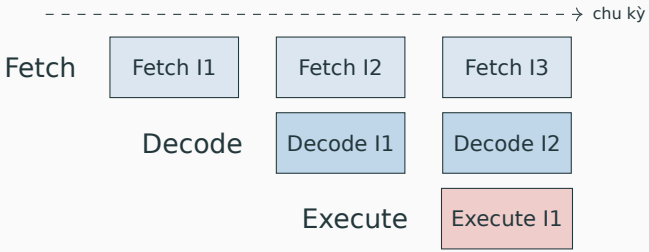
- Phần lớn lệnh 16-bit đơn giản (số học, logic, dịch bit, di chuyển) hoàn tất trong **1 chu kỳ**.
- ALU truy cập trực tiếp toàn bộ thanh ghi, không cần bước trung gian truy xuất bộ nhớ chậm – giống AVR.
- Áp dụng **pipeline 3 tầng**: khi lệnh n thực thi thì lệnh $n+1$ đã giải mã và lệnh $n+2$ đã được nạp – xem chi tiết ở tài liệu tham khảo về quy trình biên dịch.

Bảng số chu kỳ theo loại lệnh (Cortex-M3)

Loại lệnh	Số chu kỳ	Ví dụ
Số học/logic/dịch bit trên thanh ghi	1	ADDS, LSLS
Truy xuất SRAM (load/store đơn)	2	LDR, STR
Rẽ nhánh có điều kiện (nhảy/không nhảy)	1 hoặc 1–2	BNE
Nhảy/gọi hàm	2–4	BL, BX
Nhân 32-bit (single-cycle multiplier)	1	MULS

Ví dụ: ở 72 MHz, 1 chu kỳ $= 1/72\text{MHz} \approx 13{,}9$ ns $\rightarrow$ nhanh hơn nhiều so với 62,5 ns của AVR ở 16 MHz.

Kiến trúc pipeline 3 tầng



Trong khi I_1 đang *thực thi*, I_2 đang *giải mã* và I_3 đã được *lấy* – ba lệnh chồng lấn đồng thời, thêm 1 tầng so với pipeline 2 tầng của AVR.

Lợi ích và giới hạn của pipeline 3 tầng

- **Lợi ích**: thêm tầng Decode tách biệt giúp mỗi tầng đơn giản hơn, cho phép tăng xung nhịp tối đa so với gộp giải mã vào Execute.
- **Giới hạn**: khi gặp lệnh nhảy/rẽ nhánh, **2** lệnh đã ”lấy trước” có thể bị huỷ (pipeline flush) – tốn nhiều chu kỳ hơn AVR (vốn chỉ mất 1 lệnh khi flush pipeline 2 tầng).
- So với pipeline sâu (5–15 tầng) của Cortex-A hay x86: Cortex-M vẫn đơn giản, dễ dự đoán thời gian thực thi – phù hợp hệ thời gian thực.

So sánh kiến trúc Cortex-M (RISC 32-bit) với CISC truyền thống

Tiêu chí	RISC (Cortex-M)	CISC (vd x86 cũ)
Số lượng lệnh	Vừa phải, đã mở rộng (Thumb-2, hàng trăm biến thể)	Nhiều, phức tạp
Độ dài mã lệnh	Chủ yếu cố định theo cặp (16 hoặc 32-bit)	Thay đổi (1–15 byte)
Truy xuất bộ nhớ	Chỉ qua LDR/STR (load–store)	ALU thao tác trực tiếp trên bộ nhớ
Số chu kỳ/lệnh	Đa số 1 chu kỳ	Thường nhiều hơn 1, thay đổi

Cortex-M vẫn giữ triết lý RISC load–store giống AVR, chỉ mở rộng độ dài lệnh linh hoạt hơn (Thumb-2) để cân bằng giữa mật độ mã và sức mạnh biểu diễn.

Kiểm tra nhanh – Câu hỏi

Thảo luận

1. Vì sao Cortex-M cần tới **hai** con trỏ ngăn xếp (MSP, PSP) trong khi AVR chỉ cần một?
2. Sau lệnh `SUBS R0, R0, R1` nếu `R0` = `R1` thì cờ nào trong APSR sẽ bằng 1?

Kiểm tra nhanh – Gợi ý đáp án

1. MSP phục vụ chương trình chính và toàn bộ ngữ cảnh ngắt (đảm bảo ngăn xếp ngắt luôn tách biệt, an toàn); PSP dành riêng cho từng tác vụ khi chạy hệ điều hành thời gian thực (RTOS) – AVR không có khái niệm đa tác vụ ở mức phần cứng nên chỉ cần một ngăn xếp.
2. Kết quả `R0 - R1 = 0` $\rightarrow$ cờ **Zero (Z) = 1**; cờ Carry (C) = 1 (quy ước ARM: C=1 khi phép trừ **không** cần mượn, ngược với trực giác – khác quy ước AVR).

Tổng kết: Kiến trúc CPU ARM Cortex-M

- CPU Cortex-M = 32-bit ALU (+ chia, nhân phần cứng) + 16 thanh ghi (R0–R15) + xPSR (cờ APSR + IPSR + EPSR) + bộ điều khiển giải mã Thumb-2.
- Hai con trỏ ngăn xếp MSP/PSP; LR là thanh ghi riêng lưu địa chỉ trở về, khác cơ chế ngăn xếp thuần tuý của AVR.
- Pipeline 3 tầng (Fetch–Decode–Execute), đạt **1,25 DMIPS/MHz**, cao hơn nhiều so với AVR ở cùng đơn vị xung nhịp.

GPIO và định địa chỉ trên ARM Cortex-M

Không gian địa chỉ ngoại vi trong Cortex-M

- Cortex-M **không có** tập lệnh I/O chuyên dụng như AVR (`IN`/`OUT`) – toàn bộ thanh ghi ngoại vi được **ánh xạ bộ nhớ** (memory-mapped) trong vùng địa chỉ `0x40000000`–`0x5FFFFFFF` (theo bản đồ bộ nhớ ARMv7-M chuẩn).
- Truy cập thanh ghi ngoại vi dùng **cùng lệnh** `LDR`/`STR` như truy cập SRAM thông thường – không có sự phân biệt ”không gian I/O riêng” như AVR.
- STM32F103 tổ chức ngoại vi theo 2 bus `APB1` (tần số thấp) và `APB2` (tần số cao, gồm GPIO) kết nối qua Bus Matrix.

Phân cấp bus ngoại vi: AHB, APB1, APB2

- **AHB** (Advanced High-performance Bus): kết nối trực tiếp lõi CPU với bộ nhớ và các ngoại vi tốc độ cao (DMA).
- **APB2** (tần số cao, tối đa bằng xung hệ thống): GPIO, USART1, SPI1, ADC.
- **APB1** (tần số thấp hơn, tối đa bằng 1/2 xung hệ thống): USART2/3, SPI2/3, I2C, Timer đa dụng.
- Khác biệt AVR: AVR chỉ có một không gian I/O phẳng (64 thanh ghi cổ điển + Extended I/O); STM32 phân cấp bus theo tốc độ để tối ưu tiêu thụ điện và băng thông.

Cách truy cập thanh ghi ngoại vi: chỉ một con đường duy nhất

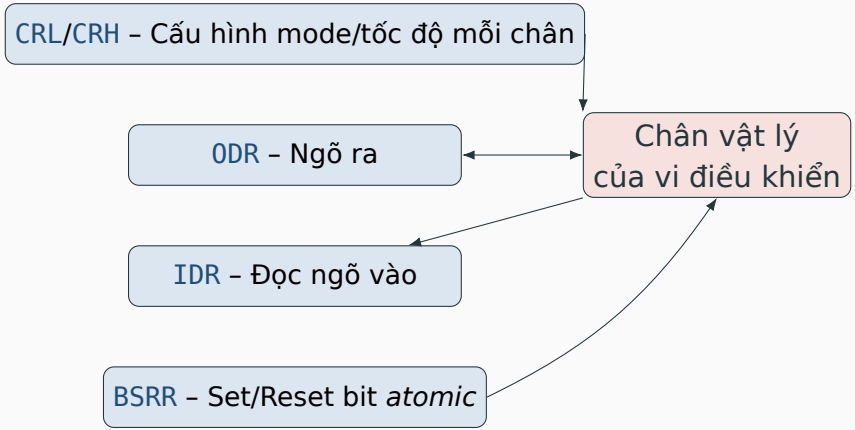
Cách truy cập	Đặc điểm	Phạm vi
LDR/STR	2 chu kỳ, địa chỉ 32-bit đầy đủ	Toàn bộ 4GB không gian địa chỉ, bao gồm mọi ngoại vi

Không tồn tại ”lệnh I/O nhanh 1 chu kỳ” như `IN/OUT` của AVR – bù lại, mọi ngoại vi đều truy cập theo **cùng một cách nhất quán**, giúp trình biên dịch C tối ưu dễ dàng hơn (không cần phân biệt vùng nhớ).

Các cổng GPIO trên STM32F103C8 (Blue Pill)

- **GPIOA, GPIOB**: mỗi cổng **16 bit** (`PA0`–`PA15`, `PB0`–`PB15`) – rộng gấp đôi mỗi cổng 8-bit của AVR.
- **GPIOC**: thường chỉ dùng `PC13`–`PC15` trên board Blue Pill (PC13 nối sẵn LED onboard).
- Tổng cộng nhiều hơn 40 chân GPIO khả dụng, so với 23 chân của ATmega328P (PORTB + PORTC + PORTD).

Bốn thanh ghi điều khiển mỗi cổng GPIO



So với AVR (chỉ 3 thanh ghi: DDRx/PORTx/PINx), STM32 có **4 thanh ghi** – thêm BSRR cho phép ghi 1 bit không cần đọc-sửa-ghi.

Thanh ghi CRL/CRH (Configuration Register Low/High)

- Mỗi chân dùng **4 bit** trong CRL (chân 0–7) hoặc CRH (chân 8–15) để chọn **đồng thời** chế độ (input/output) **và** cấu hình chi tiết (tốc độ ngõ ra, loại input).
- Vd: cấu hình `PC13` làm ngõ ra push-pull tốc độ 2MHz cần ghi đúng 4 bit tại vị trí tương ứng trong CRH.
- Phức tạp hơn thanh ghi `DDRx` một bit/chân của AVR, nhưng **gộp chung** được nhiều thuộc tính (mode + tốc độ + loại) chỉ trong một lần ghi thanh ghi.

Thanh ghi ODR (Output Data Register) khi là ngõ ra

- Khi chân được cấu hình là ngõ ra (qua CRL/CRH): bit `ODRx` quyết định mức điện áp xuất ra – **1 = mức cao**, **0 = mức thấp** – vai trò tương đương `PORTx` của AVR khi là ngõ ra.
- Đọc-sửa-ghi trực tiếp `ODR` có **rủi ro** nếu một ngắt khác cũng ghi ODR xen giữa – nên ưu tiên dùng `BSRR` (khung tiếp theo).

Thanh ghi BSRR – ghi bit an toàn (atomic)

- BSRR (Bit Set/Reset Register) 32-bit: nửa thấp (bit 0–15) **đặt** (set) bit ODR tương ứng lên 1; nửa cao (bit 16–31) **xoá** (reset) về 0.
- Ghi vào BSRR là thao tác **một lệnh, không cần đọc trước** – an toàn tuyệt đối khi có ngắt xen ngang, khác với đọc-sửa-ghi thông thường trên ODR.
- Tương tự vai trò của SBI/CBI trên AVR (thao tác 1 bit không ảnh hưởng bit khác), nhưng **an toàn hơn**: BSRR không bao giờ đọc giá trị cũ, trong khi SBI/CBI vẫn có khe hở lý thuyết trong một số kịch bản đọc-sửa-ghi phức tạp.

Thanh ghi IDR – đọc ngõ vào

- `IDR` (Input Data Register) phản ánh **mức điện áp thực tế** đo được tại chân – vai trò tương đương `PINx` của AVR.
- Đọc trạng thái nút nhấn: kiểm tra bit tương ứng, vd `if (GPIOA->IDR & (1<<0))`.
- Khác AVR (ghi 1 vào PINx sẽ đảo PORTx): IDR trên STM32 là thanh ghi **chỉ đọc**, không có hiệu ứng đảo bit khi ghi.

Bảng tổng hợp: cấu hình CRL/CRH thường dùng

Chế độ (MODE + CNF)	Giá trị 4-bit	Trạng thái chân
Input floating	`0100`	Ngõ vào, tổng trở cao, không pull
Input pull-up/pull-down	`1000`	Ngõ vào, có pull (chọn qua ODR)
Output push-pull, 2MHz	`0010`	Ngõ ra, có thể kéo cao/thấp chủ động
Output open-drain, 2MHz	`0110`	Ngõ ra, chỉ kéo thấp chủ động

Ví dụ: cấu hình PC13 output, PA0 input (thanh ghi trực tiếp)

```c
#include "stm32f1xx.h"

int main(void) {
    RCC->APB2ENR |= RCC_APB2ENR_IOPCEN | RCC_APB2ENR_IOPAEN;

    GPIOC->CRH &= ~(0xF << 20);   // xoa cau hinh cu PC13
    GPIOC->CRH |=  (0x2 << 20);   // PC13 = output push-pull 2MHz

    GPIOA->CRL &= ~(0xF << 0);    // xoa cau hinh cu PA0
    GPIOA->CRL |=  (0x8 << 0);    // PA0 = input pull-up/pull-down
    GPIOA->ODR |=  (1 << 0);      // chon pull-up cho PA0

    while (1) {
        if (GPIOA->IDR & (1 << 0))
            GPIOC->BSRR = (1 << 13);        // tat LED (active-low)
        else
            GPIOC->BSRR = (1 << (13+16));   // bat LED
    }
}
```

Ví dụ tương đương bằng thư viện STM32 HAL

- `HAL_GPIO_Init(GPIOx, &GPIO_InitStruct)` ≡ ghi các thanh ghi CRL/CRH theo cấu trúc mô tả mode/tốc độ/pull.
- `HAL_GPIO_WritePin(GPIOx, pin, state)` ≡ ghi BSRR (an toàn, atomic).
- `HAL_GPIO_ReadPin(GPIOx, pin)` ≡ đọc bit trong IDR.
- Vai trò tương đương `pinMode()`/`digitalWrite()`/`digitalRead()` của thư viện Arduino, nhưng STM32CubeMX còn **sinh sẵn mã cấu hình** từ giao diện đồ hoạ.

So sánh: thanh ghi trực tiếp và thư viện HAL

Tiêu chí	Thanh ghi trực tiếp	Thư viện HAL
Tốc độ	Nhanh nhất (ghi thẳng BSRR)	Chậm hơn (qua nhiều lớp hàm)
Dung lượng code	Nhỏ gọn	Lớn hơn đáng kể
Tính di động	Gắn với từng dòng chip cụ thể	Dễ chuyển giữa các dòng STM32
Độ khó	Cần hiểu rõ thanh ghi phần cứng	Dễ tiếp cận, có công cụ sinh mã (CubeMX)

Tổng quan các chế độ định địa chỉ trong Thumb-2

Chế độ định địa chỉ	Toán hạng	Ví dụ
Thanh ghi trực tiếp	Rd, Rn, Rm	`MOV R1,R2`
Tức thời (Immediate)	Rd, #imm	`MOVS R0,#5`
Gián tiếp qua thanh ghi cơ sở	[Rn]	`LDR R0,[R1]`
Gián tiếp có độ dời (off-set)	[Rn,#imm]	`LDR R0,[R1,#4]`
Gián tiếp có chỉ số thanh ghi	[Rn,Rm]	`LDR R0,[R1,R2]`
Tự tăng sau (post-indexed)	[Rn],#imm	`LDR R0,[R1],#4`
Tự tăng trước (pre-indexed)	[Rn,#imm]!	`LDR R0,[R1,#4]!`
Chương trình tương đối (literal pool)	PC-relative	`LDR R0,=0x12345678`

Định địa chỉ tức thời (Immediate Addressing) trên Thumb

- Toán hạng là một **hằng số** nằm ngay trong mã lệnh – lệnh 16-bit `MOVS Rd,#imm8` chỉ chứa được hằng số **8-bit** (0–255).
- Muốn nạp hằng số 32-bit đầy đủ: dùng cặp lệnh `MOVW`/`MOVT` (nạp nửa thấp rồi nửa cao) hoặc kỹ thuật *literal pool* (`LDR Rd,=hằng_số`, trình biên dịch tự đặt hằng số vào Flash gần đó rồi nạp qua PC-relative).
- Khác AVR: `LDI` của AVR chỉ giới hạn ở R16–R31; Thumb-2 `MOVS`/`MOVW` áp dụng được cho **bất kỳ** thanh ghi nào trong 16 thanh ghi.

Định địa chỉ thanh ghi trực tiếp trên Thumb

- Toán hạng nguồn và đích đều là thanh ghi, chỉ định bằng chỉ số (3 hoặc 4 bit tuỳ dạng lệnh).
- Ví dụ: `MOV Rd,Rr` (sao chép), `ADDS Rd,Rn,Rm`, `EORS Rd,Rn`.
- Nhanh nhất về giải mã, giống hệt vai trò của định địa chỉ thanh ghi trực tiếp trong AVR.

Định địa chỉ trực tiếp bộ nhớ: literal pool

- Cortex-M **không có** lệnh nạp thẳng địa chỉ SRAM 32-bit ”nguyên vẹn trong một từ lệnh” như LDS/STS của AVR (vì từ lệnh Thumb quá hẹp để chứa cả opcode và địa chỉ 32-bit).
- Giải pháp: `LDR Rd,[PC,#offset]` – đọc hằng số/địa chỉ đã được trình biên dịch đặt sẵn gần đó trong Flash (gọi là *literal pool*), định vị tương đối theo PC.
- Cú pháp `LDR Rd,=giá_trị` là bút danh (pseudo-instruction) mà trình hợp dịch tự động chuyển thành dạng PC-relative này.

Định địa chỉ gián tiếp qua thanh ghi cơ sở

- Địa chỉ SRAM cần truy cập được lưu **trong một thanh ghi bất kỳ** (không giới hạn 3 thanh ghi như X/Y/Z của AVR).
- Ví dụ: `LDR R0,[R1]` đọc từ (word) tại địa chỉ đang lưu trong `R1` vào `R0`; `STR R0,[R1]` ghi ngược lại.
- Thiết yếu cho: duyệt mảng, thao tác chuỗi, truyền tham số con trỏ trong C – vai trò tương đương định địa chỉ gián tiếp qua X/Y/Z của AVR nhưng linh hoạt hơn (dùng được bất kỳ thanh ghi nào).

Các biến thể định địa chỉ có độ dời và tự tăng/giảm

Cú pháp	Ý nghĩa	Thanh ghi cơ sở sau lệnh
`LDR R0,[R1]`	Đọc tại địa chỉ R1	Không đổi
`LDR R0,[R1,#4]`	Đọc tại địa chỉ (R1+4)	Không đổi
`LDR R0,[R1],#4`	Đọc tại R1, rồi tăng con trỏ	R1 = R1+4
`LDR R0,[R1,#4]!`	Tăng con trỏ trước, rồi đọc	R1 = R1+4

Tương ứng khái niệm với `LD Rd,X+` (tự tăng sau) và `LD Rd,-X` (tự giảm trước) của AVR, nhưng Thumb-2 cho phép độ dời (#offset) tuỳ ý thay vì chỉ 0–63 như LDD/STD.

Ví dụ: duyệt mảng SRAM bằng con trỏ thanh ghi

```
    LDR   R1, =mang        ; nap dia chi mang vao R1
    MOVS  R2, #5             ; so phan tu can duyet
    MOVS  R3, #0               ; bien tong = 0

sum_loop:
    LDR   R0, [R1], #4          ; doc phan tu, R1 tu dong +4
    ADDS  R3, R3, R0              ; cong don vao R3
    SUBS  R2, R2, #1
    BNE   sum_loop                  ; lap den khi het mang
```

Mỗi lần lặp: đọc 1 phần tử 32-bit và tự động dịch con trỏ R1 sang phần tử kế tiếp (+4 byte, vì mỗi phần tử `int` rộng 4 byte) – so với AVR chỉ +1 byte mỗi lần vì phần tử 8-bit.

So sánh tốc độ các chế độ định địa chỉ (Cortex-M)

Chế độ định địa chỉ	Số chu kỳ điển hình
Thanh ghi trực tiếp / Tức thời	1
Bộ nhớ gián tiếp (LDR/STR qua thanh ghi)	2
Gián tiếp có độ dời / tự tăng giảm	2
Literal pool (PC-relative)	2

Cùng nguyên lý với AVR: truy cập thanh ghi luôn nhanh hơn truy cập SRAM/Flash – vẫn nên giữ biến ”nóng” trong thanh ghi khi tối ưu hoá.

Kiểm tra nhanh – Câu hỏi

Thảo luận

1. Vì sao ghi vào `BSRR` lại an toàn hơn khi có ngắt xen ngang so với đọc-sửa-ghi trực tiếp trên `ODR`?
2. Chế độ định địa chỉ nào phù hợp nhất để duyệt qua một mảng 20 phần tử `int` (32-bit) trong SRAM? Vì sao bước nhảy là 4 chứ không phải 1?

Kiểm tra nhanh – Gợi ý đáp án

1. Ghi BSRR là một lệnh STR duy nhất, không cần đọc giá trị ODR hiện tại trước – nên không có ”khoảng hở” giữa đọc và ghi để một ngắt khác chen vào làm mất thay đổi (lost update); đọc-sửa-ghi trên ODR có khoảng hở này.
2. Định địa chỉ tự tăng sau (`[R1],#4`): mỗi lệnh `LDR R0,[R1],#4` vừa đọc phần tử vừa tự động dịch con trỏ sang phần tử kế tiếp; bước nhảy là 4 vì mỗi `int` chiếm 4 byte trên kiến trúc 32-bit (so với 1 byte trên AVR 8-bit).

Tổng kết: GPIO và định địa chỉ trên Cortex-M

- Mỗi cổng GPIO có 4 thanh ghi: `CRL`/`CRH` (cấu hình), `ODR` (ngõ ra), `IDR` (đọc vào), `BSRR` (ghi bit atomic).
- Không gian địa chỉ hoàn toàn phẳng (memory-mapped), không có lệnh I/O riêng như AVR.
- Thumb-2 hỗ trợ nhiều chế độ định địa chỉ linh hoạt hơn AVR: độ dời tuỳ ý, tự tăng/giảm trên **bất kỳ** thanh ghi nào trong 16 thanh ghi.

Tập lệnh Thumb-2 trên ARM Cortex-M

Tổng quan tập lệnh Thumb-2

- Tập lệnh Thumb-2 kết hợp lệnh **16-bit** (gọn, giống Thumb cổ điển) và **32-bit** (đầy đủ tính năng hơn, gần với tập lệnh ARM truyền thống) trong **cùng một luồng lệnh**.
- Tổng số biến thể lệnh nhiều hơn AVR (AVR ≈131 lệnh cố định 16/32-bit); Thumb-2 linh hoạt hơn nhưng phức tạp hơn khi học.
- Trình biên dịch (`arm-none-eabi-gcc`) tự động chọn dạng 16-bit hay 32-bit tối ưu – lập trình viên hiếm khi viết assembly tay hoàn toàn.

Định dạng lệnh (Instruction Format) Thumb-2

- Lệnh 16-bit: thường 3 thanh ghi (3 bit/thanh ghi, low registers) hoặc 1 thanh ghi + hằng số 8-bit.
- Lệnh 32-bit: mã hoá theo 2 nửa từ (half-word) 16-bit liên tiếp, cho phép chỉ định đủ 4 thanh ghi (4 bit/thanh ghi, toàn bộ R0–R15) và hằng số rộng hơn.
- CPU tự nhận diện độ dài lệnh dựa trên 5 bit cao nhất của nửa từ đầu tiên – không cần ”chế độ” riêng như chuyển đổi ARM/Thumb trên các dòng Cortex-A.

Phân loại nhóm lệnh Thumb-2 – tổng quan

Nhóm lệnh	Chức năng chính	Ví dụ
Số học & Logic	Tính toán trên thanh ghi	`ADDS`, `ANDS`
Dịch/xoay bit	Thao tác từng bit	`LSLS`, `ROR`
So sánh & rẽ nhánh	Điều khiển luồng có điều kiện	`CMP`, `BNE`
Nhảy & gọi hàm	Điều khiển luồng không điều kiện	`B`, `BL`
Load/Store	Trao đổi dữ liệu thanh ghi $\leftrightarrow$ bộ nhớ	`LDR`, `STR`
Di chuyển dữ liệu	Nạp hằng số/sao chép thanh ghi	`MOV`, `MOVW`
Điều khiển hệ thống	Điều khiển CPU, ngắt, năng lượng	`CPSIE`, `WFI`

Nhóm lệnh số học: ADD, ADC, SUB, SBC, MUL

- `ADDS Rd,Rn,Rm`: `Rd` $\leftarrow$ `Rn` + `Rm`, cập nhật cờ (hậu tố S).
- `ADCS Rd,Rn,Rm`: cộng có nhớ (Add with Carry) – dùng để cộng số nhiều từ (nhiều hơn 32-bit).
- `SUBS Rd,Rn,Rm`: trừ. SBCS: trừ có mượn (Subtract with Carry).
- `MULS Rd,Rn,Rm`: nhân 32-bit, hoàn tất trong **1 chu kỳ** – AVR cơ bản **không có** lệnh nhân phần cứng (một số dòng AVR nâng cao mới có `MUL`).

Nhóm lệnh logic: AND, ORR, EOR, BIC, MVN

- `ANDS Rd,Rn,Rm`: AND theo bit – dùng để ”xoá” các bit cụ thể.
- `ORRS Rd,Rn,Rm`: OR theo bit – dùng để ”bật” các bit cụ thể.
- `EORS Rd,Rn,Rm`: XOR theo bit – `EORS Rd,Rd` đưa thanh ghi về 0.
- `BICS Rd,Rn,Rm`: ”bit clear” – `Rd` $\leftarrow$ `Rn` AND (NOT `Rm`), gộp AND+NOT trong 1 lệnh (AVR không có lệnh tương đương trực tiếp). `MVN Rd,Rm`: đảo tất cả bit.

Ví dụ: ADDS/SUBS và ảnh hưởng lên cờ

Phép toán	Kết quả	Cờ APSR bị ảnh hưởng
`5 + 3`	`8`	Z=0, C=0, N=0, V=0
`0xFFFFFFFF + 1`	`0`	Z=1, C=1, N=0, V=0
`5 - 5`	`0`	Z=1, C=1 (không mượn), N=0, V=0
`0 - 1`	`0xFFFFFFFF`	Z=0, C=0 (có mượn), N=1, V=0

Lưu ý quy ước cờ Carry của ARM cho phép trừ **ngược** với AVR: C=1 nghĩa là **không** mượn, C=0 nghĩa là **có** mượn.

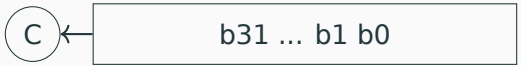
Khác biệt: Thumb-2 không có lệnh INC/DEC riêng

- AVR có lệnh chuyên dụng `INC Rd`/`DEC Rd` (không ảnh hưởng cờ Carry) để phục vụ đếm vòng lặp.
- Thumb-2 **dùng chung** `ADDS Rd,Rd,#1` / `SUBS Rd,Rd,#1` cho cả tăng/giảm lẫn cộng/trừ thông thường – đơn giản hoá tập lệnh, đổi lại luôn ảnh hưởng đầy đủ cờ APSR.
- Trình biên dịch C tự sinh đúng lệnh này khi gặp `i++` hoặc `i--` trong vòng lặp `for`.

Nhóm lệnh dịch bit: LSL, LSR, ASR

- `LSLS Rd,Rm,#n` (Logical Shift Left): dịch trái n bit, bit cao nhất dịch ra đưa vào cờ C. Tương đương nhân 2^n (không dấu).
- `LSRS Rd,Rm,#n` (Logical Shift Right): dịch phải n bit, bit 0 đưa vào cờ C. Tương đương chia 2^n (không dấu).
- `ASRS Rd,Rm,#n` (Arithmetic Shift Right): dịch phải nhưng **giữ nguyên bit dấu** – tương đương chia 2^n cho số **có dấu** – AVR không có lệnh dịch số học riêng biệt.

Nhóm lệnh xoay bit: ROR, RRX



ROR`S Rd,Rm,#n`: xoay phải n bit

- ROR`S Rd,Rm,#n`: xoay phải n bit, bit dịch ra khỏi bit 0 quay vòng lại bit 31.
- RRX `Rd,Rm`: xoay phải **1 bit qua cờ Carry** – giống hệt nguyên lý ROR của AVR (xoay qua Carry), dùng để dịch số nhiều từ.
- Khác AVR: Thumb-2 xoay được **n bit tuỳ ý** trong 1 lệnh, trong khi AVR chỉ xoay được 1 bit mỗi lệnh ROL/ROR.

Ứng dụng dịch/xoay bit trên Cortex-M

- **Nhân/chia nhanh theo luỹ thừa 2**: dịch bit rẻ hơn lệnh `MULS`/chia phần mềm khi số mũ biết trước.
- **Trích xuất trường bit**: kết hợp LSLS+LSRS để lấy ra một nhóm bit cụ thể từ thanh ghi ngoại vi (thường gặp khi đọc thanh ghi trạng thái).
- **Giao tiếp bit-banging**: tương tự AVR, nhưng ít cần thiết hơn vì Cortex-M thường có sẵn ngoại vi phần cứng UART/SPI/I2C tích hợp.

Nhóm lệnh so sánh: CMP, CMN, TST, TEQ

- `CMP Rn,Rm`: tính `Rn − Rm` **chỉ để cập nhật cờ**, không lưu kết quả – vai trò giống hệt `CP` của AVR.
- `CMN Rn,Rm`: so sánh bằng phép cộng (`Rn + Rm`) – AVR không có lệnh tương đương trực tiếp.
- `TST Rn,Rm`: AND để kiểm tra bit (không lưu kết quả) – thường dùng thay cho `SBIC`/`SBIS` của AVR khi kiểm tra 1 bit cờ.
- `TEQ Rn,Rm`: XOR để kiểm tra bằng nhau mà không ảnh hưởng cờ Carry.

Nhóm lệnh rẽ nhánh có điều kiện

Lệnh	Điều kiện nhảy	Thường dùng sau
BNE	Z = 0 (khác nhau)	CMP
BEQ	Z = 1 (bằng nhau)	CMP
BLT	Nhỏ hơn (có dấu, dựa N,V)	CMP
BGE	Lớn hơn hoặc bằng (có dấu)	CMP
BCS/BHS	Lớn hơn hoặc bằng (không dấu)	CMP
BCC/BLO	Nhỏ hơn (không dấu)	CMP

Tương ứng trực tiếp BRNE/BREQ/BRLT/BRGE của AVR; thêm cặp BCS/BCC chuyên cho so sánh không dấu (AVR gọi là BRCS/BRCC).

Ví dụ: vòng lặp dùng so sánh & rẽ nhánh

```
    MOVS  R0, #0          ; bien dem i = 0
    MOVS  R1, #10          ; gioi han lap = 10

loop:
    CMP   R0, #10            ; so sanh i voi 10
    BEQ   loop_done            ; neu i == 10 thi thoat

    ; ... than vong lap ...
    ADDS  R0, R0, #1             ; i = i + 1
    B     loop

loop_done:
    NOP
```

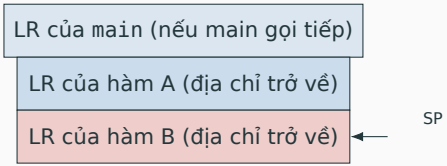
Nhóm lệnh nhảy không điều kiện: B, BX

- `B label`: nhảy **tương đối** so với PC, phạm vi tuỳ dạng lệnh (16-bit: $\pm$2K byte; 32-bit `B.W`: $\pm$16MB).
- `BX Rm`: nhảy **tuyệt đối** tới địa chỉ chứa trong thanh ghi `Rm` – thường dùng `BX LR` để trở về từ hàm.
- Khác AVR: Thumb-2 không phân biệt rạch ròi ”nhảy gần/xa” bằng 2 lệnh riêng (`RJMP`/`JMP`) – trình hợp dịch tự chọn dạng 16 hay 32-bit theo khoảng cách nhãn đích.

Nhóm lệnh gọi/trở về hàm con

- `BL label` (Branch with Link): nhảy tới hàm con, tự động ghi địa chỉ trở về vào LR.
- `BX LR`: trở về từ hàm con bằng cách nhảy tới địa chỉ trong LR.
- `POP {PC}`: cách trở về **thay thế** khi hàm có lồng gọi hàm khác (đã `PUSH {LR}` trước đó) – lấy trực tiếp địa chỉ trở về từ ngăn xếp vào PC, tương đương RET của AVR nhưng hiện thực qua lệnh POP thông thường chứ không phải lệnh chuyên dụng.

Ngăn xếp trong lời gọi hàm lồng nhau (Cortex-M)



Hàm A gọi hàm B bằng BL: vì LR sẽ bị ghi đè, hàm A phải tự PUSH {LR} trước khi gọi B, rồi POP {PC} khi kết thúc – LIFO giống AVR nhưng chỉ hàm **có gọi hàm khác** mới cần dùng ngăn xếp cho địa chỉ trở về.

Thao tác bit đơn: bit-banding và BFI/UBFX

- Không có lệnh 1-chu-kỳ thao tác trực tiếp thanh ghi I/O như SBI/CBI của AVR – thay vào đó Cortex-M (M3/M4) hỗ trợ **bit-banding**: ánh xạ mỗi bit của một vùng nhớ vào một địa chỉ word riêng trong vùng *alias*, cho phép đọc/ghi 1 bit bằng lệnh LDR/STR thông thường.
- BFI/UBFX (Bit Field Insert/Extract, Thumb-2 32-bit): chèn/trích một nhóm bit liên tiếp vào/từ thanh ghi trong 1 lệnh.
- Với GPIO cụ thể, cách thực dụng nhất vẫn là dùng BSRR (đã học ở mục trước) thay vì bit-banding.

Kiểm tra bit và rẽ nhánh (thay thế SBIC/SBIS)

- AVR có lệnh chuyên dụng `SBIC/SBIS` để ”bỏ qua lệnh kế nếu bit = 0/1”.
- Thumb-2 **không có** lệnh bỏ-qua tương đương – thay vào đó dùng tổ hợp `TST Rn,#mask` (kiểm tra bit) rồi `BEQ/BNE` (rẽ nhánh có điều kiện) – tốn 2 lệnh thay vì 1, nhưng linh hoạt hơn (kiểm tra được nhiều bit cùng lúc qua mask).
- Một số dòng Cortex-M có khối lệnh `IT` (If-Then) cho phép thực thi **có điều kiện** tối đa 4 lệnh liên tiếp mà không cần rẽ nhánh thật sự – kỹ thuật không tồn tại trên AVR.

Ví dụ: chờ trạng thái nút nhấn (TST + rẽ nhánh)

```
wait_press:
    LDR   R0, [R1]        ; doc IDR cua GPIOA vao R0
    TST   R0, #1              ; kiem tra bit 0 (PA0)
    BNE   wait_press            ; PA0 = 1 (chua nhan) -> tiep tuc cho

    ; toi day nghia la nut da duoc nhan (PA0 = 0)
    LDR   R2, =0x2000            ; bit set cho PC13 trong BSRR
    STR   R2, [R3]                 ; bat LED qua BSRR
```

So với AVR (1 lệnh SBIC + RJMP), Thumb-2 cần 3 lệnh (LDR+TST+BNE) vì không có lệnh kiểm tra bit trực tiếp trên thanh ghi ngoại vi.

Nhóm lệnh di chuyển: MOV, MOVW, MOVT

- `MOVS Rd,Rm`: sao chép nội dung 1 thanh ghi 32-bit.
- `MOVW Rd,#imm16`: nạp hằng số 16-bit vào **nửa thấp** của thanh ghi (nửa cao = 0).
- `MOVT Rd,#imm16`: nạp hằng số 16-bit vào **nửa cao**, giữ nguyên nửa thấp – kết hợp `MOVW+MOVT` để nạp trọn hằng số 32-bit bất kỳ trong đúng 2 lệnh.
- AVR không cần cặp lệnh này vì thanh ghi chỉ 8-bit, `LDI` nạp trọn trong 1 lệnh.

Nhóm lệnh truy xuất bộ nhớ: LDR/STR

- LDR/STR Rd,[Rn]: đọc/ghi **word** 32-bit.
- LDRB/STRB: đọc/ghi **byte** 8-bit (tương đương LD/ST của AVR về độ rộng).
- LDRH/STRH: đọc/ghi **half-word** 16-bit – AVR không có khái niệm half-word vì thanh ghi vốn chỉ 8-bit.
- Đây là nhóm lệnh **duy nhất** được phép truy cập bộ nhớ dữ liệu, đúng triết lý RISC load–store – giống hệt nguyên lý của AVR.

Bảng tổng hợp: lệnh di chuyển dữ liệu

Lệnh	Cú pháp	Số chu kỳ
MOV	MOVS Rd,Rm	1
MOVW/MOVT	MOVW Rd,#imm16	1
LDR/STR	LDR Rd,[Rn]	2
LDRB/STRB	LDRB Rd,[Rn]	2
LDRH/STRH	LDRH Rd,[Rn]	2
PUSH/POP	PUSH {Rlist}	$1+n$ (n thanh ghi)

Nhóm lệnh điều khiển hệ thống

- `NOP`: không làm gì, tốn đúng 1 chu kỳ – giống AVR.
- `CPSIE I` / `CPSID I`: bật/tắt toàn bộ ngắt (thao tác cờ PRIMASK) – tương đương SEI/CLI của AVR.
- `WFI` (Wait For Interrupt) / `WFE` (Wait For Event): đưa CPU vào chế độ ngủ chờ ngắt/sự kiện – tương đương vai trò SLEEP của AVR nhưng có thêm biến thể theo sự kiện (event).
- Không có lệnh `WDR` riêng – watchdog trên STM32 được điều khiển qua thanh ghi ngoại vi `IWDG`/`WWDG` thay vì một lệnh CPU chuyên dụng.

Ví dụ chương trình đầy đủ – Cộng hai số

```
    .syntax unified
    .thumb

    .text
    .global main
main:
    MOVS  R0, #30          ; so hang thu nhat = 30
    MOVS  R1, #12           ; so hang thu hai = 12
    ADDS  R0, R0, R1          ; R0 = R0 + R1 = 42

    LDR   R2, =0x40011000       ; dia chi GPIOA base
    MOVS  R3, #0xFF               ; gia tri cau hinh minh hoa
    STR   R3, [R2]                  ; xuat ket qua (minh hoa)

end:
    B     end                         ; dung tai day
```

Ví dụ: vòng lặp đếm giảm dần

```
    MOVS  R0, #200         ; bien dem = 200
    LDR   R1, =0x4001100C    ; dia chi thanh ghi ODR (minh hoa)

count_loop:
    STR   R0, [R1]              ; xuat gia tri dem hien tai
    SUBS  R0, R0, #1              ; giam bien dem
    BNE   count_loop                ; lap lai neu R0 != 0

stop:
    B     stop                        ; dung tai day
```

Bảng thời gian thực thi lệnh theo nhóm (Cortex-M3)

Nhóm lệnh	Số chu kỳ	Ví dụ
Số học/logic/dịch bit trên thanh ghi	1	`ADDS`, `LSLS`
Di chuyển thanh ghi, tức thời	1	`MOV`, `MOVS`
Nhân 32-bit	1	`MULS`
Truy xuất bộ nhớ (load/store đơn)	2	`LDR`, `STR`
Rẽ nhánh có điều kiện	1–2	`BNE`
Nhảy/gọi hàm	2–4	`B`, `BL`
Trở về hàm (POP nhiều thanh ghi)	$1 + n$	`POP`

Ước lượng hiệu năng thực tế

Bài toán

Vòng lặp `count_loop` ở ví dụ trước (STR + SUBS + BNE) lặp 200 lần ở tần số 72 MHz. Ước lượng thời gian thực thi.

- 1 chu kỳ $= 1/72.10^{6} \approx 13{,}9$ ns.
- Mỗi vòng lặp: STR (2) + SUBS (1) + BNE (1–2, giả sử nhảy) ≈ 5 chu kỳ.
- Tổng: $200 \times 5 = 1000$ chu kỳ $\times 13{,}9$ ns $\approx 13{,}9\,\mu$s – nhanh hơn khoảng 3,6 lần so với vòng lặp tương tự trên AVR (50 μs), dù số chu kỳ/vòng lặp không giảm nhiều – lợi thế chủ yếu đến từ xung nhịp cao hơn.

Quan hệ mã C và assembly do compiler sinh ra

Mã C

```c
uint32_t a = 5, b = 3, c;
c = a + b;
```

Assembly (minh hoạ)

```
MOVS  R0, #5
MOVS  R1, #3
ADDS  R2, R0, R1
```

`arm-none-eabi-gcc` cấp phát thanh ghi tự động, có xu hướng dùng lệnh 16-bit gọn khi đủ điều kiện (`-Os`) hoặc 32-bit khi tối ưu tốc độ (`-O2`/`-O3`).

Ví dụ đối chiếu: vòng lặp for

```
for (i = 0; i < 10; i++) {
    GPIOA->ODR = i;
}
```

```
    MOVS  R0, #0        ; i = 0
loop:
    STR   R0, [R1]         ; ODR = i (R1 tro san toi dia chi ODR)
    ADDS  R0, R0, #1          ; i++
    CMP   R0, #10               ; so sanh i voi 10
    BNE   loop                    ; lap neu i != 10
```

So sánh: Assembly viết tay và trình biên dịch C

Tiêu chí	Assembly viết tay	C qua trình biên dịch
Kiểm soát phần cứng	Tối đa, từng chu kỳ	Gián tiếp qua trình biên dịch
Tốc độ phát triển	Chậm, dễ lỗi	Nhanh hơn nhiều
Hiệu năng	Có thể tối ưu tối đa (thủ công)	Rất tốt với -02, trình biên dịch ARM GCC tối ưu tốt
Khả năng bảo trì	Khó đọc, khó mở rộng	Dễ đọc, dễ tái sử dụng

Thực tế: hầu như toàn bộ firmware STM32 viết bằng C (qua HAL hoặc thanh ghi trực tiếp); assembly Thumb chỉ dùng cho startup code và các đoạn cực kỳ nhạy thời gian.

Kiểm tra nhanh – Câu hỏi

Thảo luận

1. Vì sao Thumb-2 cần cặp lệnh `MOVW`/`MOVT` để nạp một hằng số 32-bit, trong khi AVR chỉ cần 1 lệnh `LDI` để nạp hằng số 8-bit?
2. Cho đoạn C: `while (i != 0) i--;` – đoán 2 lệnh assembly Thumb tương ứng cho thân vòng lặp.

Kiểm tra nhanh – Gợi ý đáp án

1. Vì từ lệnh Thumb quá hẹp (16 hoặc 32-bit) để nhúng nguyên vẹn một hằng số 32-bit **cùng** với opcode và thanh ghi đích trong 1 lệnh; AVR nạp hằng số 8-bit vừa đủ trong phần còn lại của từ lệnh 16-bit nên chỉ cần 1 lệnh `LDI`.
2. Tương ứng `SUBS R0, R0, #1` (giảm `i`) và `BNE loop` (quay lại nếu `i` chưa về 0) – đúng mẫu hình SUBS+BNE, tương tự `DEC+BRNE` đã học ở AVR.

Tổng kết: Tập lệnh Thumb-2

- Tập lệnh Thumb-2 chia thành các nhóm tương tự AVR: số học/logic, dịch/xoay bit, so sánh/rẽ nhánh, nhảy/gọi hàm, load/store, di chuyển dữ liệu, điều khiển hệ thống.
- Khác biệt cốt lõi: độ dài lệnh **hỗn hợp** 16/32-bit, có bộ nhân/chia phần cứng, không có lệnh bỏ-qua kiểu SBIC/SBIS, dùng BSRR thay cho SBI/CBI.
- Hiểu tập lệnh Thumb-2 giúp đọc được mã assembly do `arm-none-eabi-gcc` sinh ra, phục vụ debug và tối ưu hoá trên STM32.

Thực hành — ARM Cortex-M (STM32)

Mục tiêu buổi thực hành (ARM Cortex-M)

- Đấu nối và đọc đúng trạng thái một nút nhấn trên board STM32, sử dụng chân GPIO cấu hình pull-up/pull-down.
- Viết được chương trình đọc nút nhấn bằng cả thanh ghi trực tiếp và thư viện HAL.
- Hoàn thành bài Lab: LED sáng theo trạng thái nút nhấn, có áp dụng chống dội phím – **cùng yêu cầu chức năng** như Lab AVR, nhưng hiện thực trên kiến trúc 32-bit.
- Làm quen công cụ debug tích hợp của STM32CubeIDE: breakpoint, theo dõi thanh ghi qua ST-Link/SWD.

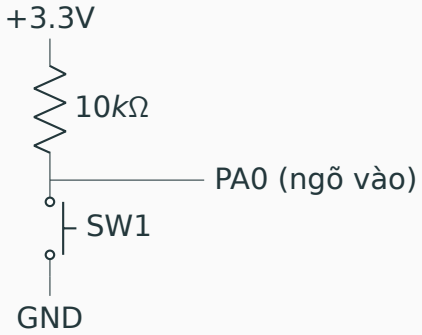
Chuẩn bị: linh kiện cần thiết (ARM Cortex-M)

- Board STM32F103C8T6 (”Blue Pill”) hoặc STM32 Nucleo, mạch nạp ST-Link V2.
- Breadboard và dây cắm (jumper wires) – giống bộ dụng cụ đã dùng ở Lab AVR.
- 1 nút nhấn (push button 4 chân), 1 LED, điện trở 330Ω (giới hạn dòng LED).
- Điện trở 10kΩ (tuỳ chọn, nếu không dùng pull-up nội của vi điều khiển).

Kết nối board STM32 với máy tính

- Cắm mạch nạp ST-Link V2 vào board qua 4 chân SWD (`SWCLK`, `SWDIO`, `GND`, `3V3`) – khác AVR (nạp qua cổng USB–Serial tích hợp sẵn trên board Arduino).
- Mở STM32CubeIDE, tạo project mới, chọn đúng dòng chip (`STM32F103C8`).
- STM32CubeMX (tích hợp trong CubeIDE) cho phép cấu hình chân GPIO bằng giao diện đồ hoạ, tự sinh code khởi tạo – không có công cụ tương đương trực tiếp trong luồng làm việc Arduino IDE cho AVR.

Sơ đồ đấu nối nút nhấn với điện trở kéo lên



Khi chưa nhấn: PA0 đọc mức **cao**. Khi nhấn: PA0 nối xuống GND, đọc mức **thấp**. Lưu ý: GPIO của STM32 hoạt động ở mức **3.3V**, không phải 5V như AVR – không được cấp trực tiếp 5V vào chân GPIO.

Điện trở kéo lên: ngoài (external) và nội (internal) trên STM32

Tiêu chí	Pull-up ngoài	Pull-up nội (CRL/CRH + ODR)
Linh kiện thêm	Cần điện trở rời (vd 10kΩ)	Không cần
Cấu hình	Cố định trên mạch	Bật/tắt bằng phần mềm (mode input pull-up/pull-down)
Giá trị điện trở	Tuỳ chọn chính xác	Cố định trong chip (≈30–50kΩ, tuỳ dòng chip)

Trong bài Lab hôm nay, ta dùng **pull-up nội** (cấu hình `CRL` = input pull-up/pull-down, `ODR` bit tương ứng = 1) để đơn giản hoá mạch – cùng nguyên lý như Lab AVR.

Đọc trạng thái nút nhấn – Code thanh ghi trực tiếp

```c
#include "stm32f1xx.h"

int main(void) {
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_IOPCEN;

    GPIOA->CRL &= ~(0xF << 0);
    GPIOA->CRL |=  (0x8 << 0);   // PA0 input pull-up/pull-down
    GPIOA->ODR |=  (1 << 0);     // chon pull-up

    GPIOC->CRH &= ~(0xF << 20);
    GPIOC->CRH |=  (0x2 << 20);  // PC13 output push-pull

    while (1) {
        GPIOC->ODR = GPIOA->IDR & 0x1;  // sao chep bit PA0 sang PC13
    }
}
```

Đọc trạng thái nút nhấn – Code HAL

```c
#include "stm32f1xx_hal.h"

int main(void) {
    HAL_Init();
    GPIO_InitTypeDef gpio = {0};

    __HAL_RCC_GPIOA_CLK_ENABLE();
    gpio.Pin = GPIO_PIN_0;
    gpio.Mode = GPIO_MODE_INPUT;
    gpio.Pull = GPIO_PULLUP;
    HAL_GPIO_Init(GPIOA, &gpio);

    while (1) {
        GPIO_PinState s = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0);
        HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, s);
    }
}
```

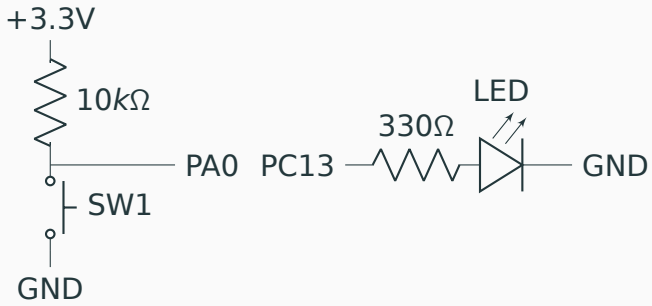
Lab: LED sáng khi nhấn nút – Đề bài (STM32)

Yêu cầu

Nối nút nhấn vào chân PA0 (dùng pull-up nội) và LED vào chân PC13 (qua điện trở 330Ω, lưu ý PC13 trên board Blue Pill thường **active-low**). Viết chương trình: **khi nhấn nút thì LED sáng, khi thả nút thì LED tắt**.

- Có thể dùng thanh ghi trực tiếp (`CRL`/`ODR`/`IDR`/`BSRR`) hoặc thư viện HAL (`HAL_GPIO_Init()`, `HAL_GPIO_ReadPin()`, `HAL_GPIO_WritePin()`).
- Nộp báo cáo: mã nguồn (C) + video/ảnh chụp kết quả hoạt động – cùng định dạng với báo cáo Lab AVR.

Lab: LED sáng khi nhấn nút – Sơ đồ đấu nối (STM32)



Nút nhấn (pull-up nội) bên trái; LED giới hạn dòng bởi điện trở 330Ω bên phải – cùng bố cục mạch như Lab AVR, chỉ đổi chân và mức điện áp (3.3V thay vì 5V).

Lab: LED sáng khi nhấn nút – Code mẫu (HAL)

```c
#include "stm32f1xx_hal.h"

int main(void) {
    HAL_Init();
    GPIO_InitTypeDef g = {0};
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOC_CLK_ENABLE();

    g.Pin = GPIO_PIN_0; g.Mode = GPIO_MODE_INPUT; g.Pull = GPIO_PULLUP;
    HAL_GPIO_Init(GPIOA, &g);
    g.Pin = GPIO_PIN_13; g.Mode = GPIO_MODE_OUTPUT_PP;
    HAL_GPIO_Init(GPIOC, &g);

    while (1) {
        if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_RESET)
            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET); // bat LED (
                active-low)
        else
            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);   // tat LED
    }
```

Hiện tượng dội phím và kỹ thuật chống dội trên STM32

- **Dội phím (switch bounce)**: hiện tượng vật lý giống hệt AVR – tiếp điểm cơ khí rung nhiều lần trước khi ổn định.
- **Chống dội bằng phần mềm**: dùng `HAL_Delay(20)` (tương đương `_delay_ms(20)` của AVR) sau khi phát hiện thay đổi trạng thái, rồi đọc lại để xác nhận.
- **Chống dội bằng phần cứng**: giống AVR (tụ lọc, Schmitt trigger); ngoài ra STM32 còn có thể dùng **Timer phần cứng** kết hợp ngắt để chống dội mà không chặn (block) vòng lặp chính – kỹ thuật nâng cao sẽ học ở Tuần 5.

Sử dụng debugger tích hợp trong STM32CubeIDE

- STM32CubeIDE gỡ lỗi **trực tiếp trên phần cứng thật** qua ST-Link/SWD – khác Microchip Studio Simulator của AVR (vốn mô phỏng thuần phần mềm, không cần board thật).
- Cho phép quan sát giá trị thanh ghi CPU (R0–R15, xPSR), thanh ghi ngoại vi (CRL, ODR, IDR...) và biến toàn cục theo thời gian thực (*Live Expressions*) trong khi chương trình đang chạy.
- Hữu ích để kiểm tra đúng giá trị thanh ghi GPIO thực tế, không chỉ mô phỏng lý thuyết như AVR Simulator.

Debug từng bước và theo dõi thanh ghi (STM32CubeIDE)

- **Step Into** (F5): thực thi từng lệnh một, *đi vào bên trong* lời gọi hàm/hàm con – giống Microchip Studio.
- **Step Over** (F6): thực thi từng dòng, chạy trọn vẹn hàm con mà *không* đi sâu vào bên trong.
- **Breakpoint** (Ctrl+Shift+B hoặc double-click lề trái): đặt điểm dừng tại một dòng lệnh cụ thể.
- Cửa sổ **Registers**/**Peripherals**: theo dõi trực tiếp thanh ghi `R0`–`R15`, `xPSR`, và toàn bộ thanh ghi ngoại vi GPIO/RCC theo tên trường bit – chi tiết hơn cửa sổ Processor Status của AVR.

Sử dụng mạch nạp / debugger thực tế: ST-Link

- **ST-Link V2**: mạch nạp/debug chính thức của ST, kết nối qua **SWD 2 dây** (`SWCLK`, `SWDIO`) – gọn hơn nhiều so với đầu nối ISP 6 chân của AVR.
- Cổng vật lý mà ST-Link kết nối vào chính là `DAP` (Debug Access Port) đã trình bày trong tài liệu tham khảo về kiến trúc lõi Cortex-M3 – ”cửa ngõ” duy nhất cho mọi thao tác nạp/gỡ lỗi.
- Cho phép đặt breakpoint, xem thanh ghi **ngay trên chip thật** với độ trễ rất thấp – tương tự vai trò của Atmel-ICE/USBasp bên AVR, nhưng SWD là giao thức chuẩn chung cho toàn bộ họ Cortex-M (không cần chọn giao thức riêng theo từng dòng chip như debugWIRE/JTAG của AVR).

Bài tập về nhà (ARM Cortex-M)

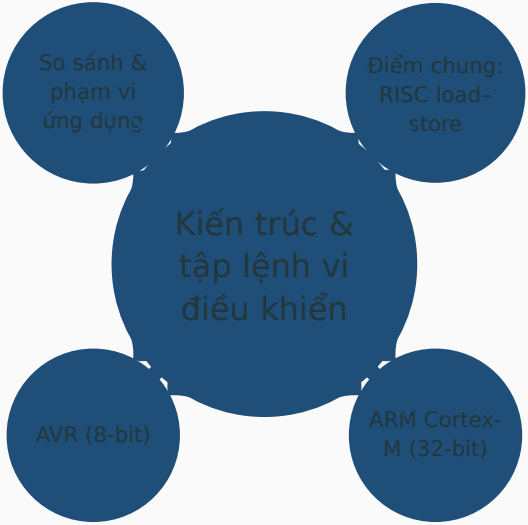
1. Đọc thêm 1 nút nhấn thứ hai (PA1), điều khiển thêm 1 LED thứ hai (PB0) độc lập với LED thứ nhất.
2. Viết lại chương trình Lab bằng thanh ghi trực tiếp thay vì HAL; so sánh dung lượng file `.bin` sinh ra giữa 2 cách viết.
3. Thêm kỹ thuật chống dội phím (debounce) bằng `HAL_Delay(20)` vào chương trình Lab, quan sát sự khác biệt khi nhấn nhả liên tục nhanh – so sánh với bài tập tương tự đã làm ở Lab AVR.

Tiêu chí đánh giá buổi thực hành (ARM Cortex-M)

Tiêu chí	Điểm
Đọc đúng trạng thái nút nhấn (thanh ghi hoặc HAL)	2
Bài Lab LED theo nút nhấn chạy đúng yêu cầu	4
Có áp dụng chống dội phím (de-bounce)	2
Báo cáo rõ ràng, đầy đủ	1
Hoàn thành đúng thời gian	1

Tổng kết & So sánh: AVR và ARM Cortex-M

Bản đồ tư duy – Hai họ kiến trúc đã học



So sánh tổng quan kiến trúc

Tiêu chí	AVR (vd ATmega328P)	ARM Cortex-M (vd STM32F103)
Độ rộng dữ liệu	8-bit	32-bit
Kiến trúc	RISC load–store, Harvard biến thể	RISC load–store, Harvard biến thể (ARMv7-M)
Tần số xung nhịp điển hình	16 MHz	72 MHz
Hiệu năng	≈16 MIPS	≈1,25 DMIPS/MHz (≈90 DMIPS ở 72MHz)
Bộ nhân/chia phần cứng	Không (cơ bản)	Có (từ Cortex-M3)

So sánh tập thanh ghi

Tiêu chí	AVR	ARM Cortex-M
Số thanh ghi đa dụng	32 x 8-bit (R0–R31)	13 x 32-bit (R0–R12)
Con trỏ ngăn xếp	1 (SPH:SPL)	2 (MSP, PSP)
Địa chỉ trở về hàm	Lưu tự động trong ngăn xếp	Lưu trong thanh ghi LR (R14)
Thanh ghi trạng thái	SREG (8 cờ: C,Z,N,V,S,H,T,I)	xPSR (APSR+IPSR+EPSR)

So sánh cơ chế ngắt và GPIO

Tiêu chí	AVR	ARM Cortex-M
Bộ điều khiển ngắt	Đơn giản, ưu tiên cố định theo vector	NVIC: lồng ngắt, ưu tiên lập trình được
Lưu ngữ cảnh khi ngắt	Phần mềm tự PUSH thủ công	Phần cứng tự động (8 thanh ghi)
Thanh ghi GPIO/cổng	3 (DDRx/PORTx/PINx)	4 (CRL/CRH, ODR, IDR, BSRR)
Ghi 1 bit an toàn	SBI/CBI (2 chu kỳ)	BSRR (atomic, 2 chu kỳ)
Không gian truy cập	I/O riêng + bộ nhớ (2 cách)	Chỉ bộ nhớ (memory-mapped, 1 cách)

So sánh tập lệnh

Tiêu chí	AVR	Thumb-2 (Cortex-M)
Độ dài lệnh	Cố định 16-bit (một số 32-bit)	Hỗn hợp 16/32-bit
Số lượng lệnh	≈131	Nhiều hơn, đa dạng hơn
Lệnh tăng/giảm riêng	Có (`INC`/`DEC`)	Không (dùng `ADDS`/`SUBS #1`)
Kiểm tra bit đơn	`SBIC`/`SBIS` (1 lệnh)	`TST`+rẽ nhánh (2 lệnh)
Nạp hằng số lớn	`LDI` (8-bit, 1 lệnh, chỉ R16–R31)	`MOVW`/`MOVT` (32-bit, 2 lệnh, mọi thanh ghi)

So sánh hiệu năng và tiêu thụ điện

Tiêu chí	AVR	ARM Cortex-M
Thông lượng đỉnh	≈16 MIPS @16MHz	≈90 DMIPS @72MHz (Cortex-M3)
Chế độ tiết kiệm điện	SLEEP (vài mức)	Nhiều mức (Sleep, Stop, Standby) + WIC hỗ trợ đánh thức khi NVIC tắt
Giá thành chip điển hình	Rất thấp	Thấp – trung bình (nhỉnh hơn AVR cùng phân khúc)
Độ phức tạp lập trình	Thấp, dễ tiếp cận	Trung bình – cao hơn (nhiều thanh ghi cấu hình, nhiều bus)

So sánh công cụ phát triển

Tiêu chí	AVR	ARM Cortex-M
IDE phổ biến	Arduino IDE, Microchip Studio	STM32CubeIDE, Keil, IAR
Trình biên dịch	avr-gcc	arm-none-eabi-gcc
Công cụ sinh code cấu hình	Không có (Arduino chỉ có thư viện)	STM32CubeMX (đồ hoạ, sinh code tự động)
Mạch nạp/debug	USB–Serial (boot-loader) hoặc AVR ISP/Atmel-ICE	ST-Link/V2 qua SWD

Điểm giống nhau cốt lõi giữa AVR và ARM Cortex-M

- Cả hai đều là kiến trúc **RISC load–store**: ALU chỉ thao tác trên thanh ghi, mọi truy cập bộ nhớ đi qua lệnh LD/ST (AVR) hay LDR/STR (Cortex-M) chuyên biệt.
- Cả hai dùng kiến trúc **Harvard biến thể**: bus lệnh và bus dữ liệu tách biệt vật lý, giúp nạp lệnh và truy cập dữ liệu diễn ra song song.
- Cả hai đều thực thi lệnh qua chu trình **Fetch–Decode–Execute** theo pipeline (2 tầng ở AVR, 3 tầng ở Cortex-M) để đạt CPI gần 1.
- Cả hai đều có quy trình biên dịch C tương tự nhau: mã nguồn $\rightarrow$ file đối tượng $\rightarrow$ liên kết $\rightarrow$ nạp vào Flash – chỉ khác công cụ (`avr-gcc` vs `arm-none-eabi-gcc`) và chi tiết thanh ghi/địa chỉ.

Phạm vi ứng dụng: khi nào chọn AVR, khi nào chọn Cortex-M?

- **Ưu tiên AVR** khi: bài toán đơn giản (điều khiển vài GPIO, cảm biến rời rạc), ngân sách cực thấp, không cần xử lý số thực/toán phức tạp, đội ngũ mới học nhúng cần mô hình dễ hình dung.
- **Ưu tiên ARM Cortex-M** khi: cần xử lý tín hiệu số (lọc, FFT), giao tiếp nhiều ngoại vi tốc độ cao đồng thời (USB, Ethernet, nhiều UART/SPI/I2C), cần chạy hệ điều hành thời gian thực (RTOS) nhờ có MPU và hai ngăn xếp MSP/PSP, hoặc cần bộ nhớ/tốc độ lớn hơn nhiều.
- Trong thực tế công nghiệp: AVR (và các dòng 8-bit tương tự) dần được thay thế bởi Cortex-M ngay cả ở phân khúc giá thấp, nhưng AVR vẫn phổ biến trong giáo dục và các thiết bị cực kỳ đơn giản, tiêu thụ điện cực thấp.

Kết luận Tuần 2: AVR và ARM Cortex-M

Tổng kết

Hai họ kiến trúc AVR (8-bit) và ARM Cortex-M (32-bit) minh hoạ rõ nét cùng một triết lý RISC load–store nhưng ở hai mức độ phức tạp khác nhau – nắm vững cả hai giúp sinh viên hiểu được sự **đánh đổi thiết kế** (độ rộng dữ liệu, số thanh ghi, độ phức tạp ngoại vi) khi chọn vi điều khiển cho một bài toán nhúng cụ thể.

Nội dung chi tiết hơn về NVIC, bản đồ bộ nhớ, quy trình biên dịch/khởi động của Cortex-M đã được trình bày trong tài liệu tham khảo riêng *”Kiến trúc ARM Cortex-M & Quy trình biên dịch”*.

Hết Tuần 2

Hẹn gặp lại ở Tuần 3: Lập trình C cho hệ thống nhúng